Concept: Cloud-Native, Distributed Document Management System (DMS)

Version: 0.18 (Draft)
Status: Concept Phase

---

1. Target Vision & Guiding Principles

The system is an audit-proof, highly configurable document management system operated as a distributed microservice architecture. Central guiding principles:

1. **Extensibility through "adding on"**: New functionality (connectors, processing steps, UI modules) is realized by adding new plugin/service instances, not by modifying existing code. Every new instance registers itself independently with the Registry Service and is automatically placed on the available server landscape by the Plugin Orchestration Service (3.8).
2. **Separation of data flow and control flow**: The Registry Service controls *who is allowed to reach whom and whether they are permitted to* (control flow including license status), while the actual document/data traffic runs directly between the participating services.
3. **Configuration instead of code**: Object definitions, conditions, permission structures, workflows can be imported and exported via JSON configuration - systems should be cloneable, or their configuration transferable.
4. **Auditability as an architectural principle, not a feature**: Every write and read action generates an immutable audit event. This is not logging bolted on after the fact, but structurally anchored in every service.
5. **Schema-separated shared database**: A single shared database instance (operationally simpler: backups, scaling, operations), but strict schema separation per service - no service reads or writes outside its own schema.
6. **Functional completeness compared to established market solutions, with genuine horizontal scaling as a differentiator**: The system's core functional scope should keep pace with established products such as nscale or VIS Suite (see section 12 for a detailed market comparison). Where their architecture is structurally limited - for instance, a core

architecture at classic EIM platforms such as nscale that is not consistently horizontally scalable - the concept described here should offer a genuine advantage through consistent stateless scalability of every service (in conjunction with the Plugin Orchestration Service, 3.8), rather than bolting scaling on afterwards. This same statelessness is also the foundation for uninterrupted rolling updates (10.5) - an operational advantage that a retrofitted, scaled architecture can only reproduce with difficulty.

1a. Backend Technology Choice (Python-first, polyglot-capable)

To start, **Python** is used as the implementation language for the backend services as far as possible. This is deliberately compatible with the architectural principle from 3.5/3.2: since all services communicate with each other exclusively via APIs (no shared in-process code, no language-specific coupling), any individual component can later be replaced without loss by a faster/different language as soon as this proves necessary for a specific service (e.g. a particularly throughput-critical rendering or storage backend service in Go or Rust) - **Python is the pragmatic starting point, not a permanent commitment for the entire system**.

Recommended Python stack

- **Web/API framework**: FastAPI (async-first, native OpenAPI schema generation - can be used directly for the plugin/connector manifests from 3.3/3.8, Pydantic v2 for validation).
- **Database access**: SQLAlchemy 2.0 in async mode with asyncpg as the Postgres driver - the current production standard in the Python ecosystem for exactly this combination.
- **Event bus connection**: `nats-py` (official asyncio-capable NATS client) for the default setting NATS JetStream (3.4); for the Kafka alternative, either `confluent-kafka-python` (C bindings, very performant) or `aiokafka` (purely asyncio-native, fits more consistently with the rest of the async stack).
- **Auth connection**: `Authlib` or `python-jose` for validating the OIDC/JWT tokens issued by Keycloak in every service (stateless verification without querying the Auth Service, see 4.4); `python-keycloak` for administrative interaction with Keycloak (user/group synchronization) from the Auth Service.
- **Storage backend connection** (elaborating on 3.6): `aioboto3` / `boto3` for S3-compatible

backends (works against AWS S3, MinIO, Ceph RGW, and other S3-compatible providers with the same library - fitting the requirement to address multiple S3 providers in parallel), `azure-storage-blob` for Azure Blob Storage, for the NFS backend regular Python filesystem access (`os` / `pathlib`) supplemented with platform-specific file locking (`fcntl` on Linux) - particular care is needed here with NFS locking semantics (NFSv4.1+ recommended, see also 3.6).

- **Query console language implementation** (elaborating on 6.1): since the query dialect closely follows psql and the wish is to extend an existing parser rather than write a new one, **pglast** is a good fit - an actively maintained Python module that exposes the real PostgreSQL parser (via libpg_query) as a complete, editable AST in Python, including visitor/printer infrastructure for targeted extension with the DMS-specific additional constructs (trace/hierarchy operators). This means real PostgreSQL grammar is parsed instead of an imitation, which considerably improves compatibility and maintenance effort.
- **BPMN engine** (elaborating on 7.1): **SpiffWorkflow** - a pure Python workflow/BPMN library maintained for over ten years, with explicit support for exactly the BPMN elements needed here: pools/lanes (department representation), multi-instance tasks, sub-workflows, timer/boundary events (basis for the per-step SLA time monitoring, 7.1), and signal/message events (basis for the federated process handover via the Federation Hub, 7.4). For graphical modeling, `bpmn-js-spiffworkflow` already exists as a matching extension of the widely used BPMN.js editor, which can be integrated into the Process Designer (7.1/8) instead of developing a proprietary BPMN editor from scratch.
- **OCR engine** (elaborating on 3.9): **PaddleOCR** (Apache 2.0) as the default - considerably more robust than classic Tesseract for tables, multi-column layouts, and multilingual documents; **Tesseract** (`pytesseract`) as a resource-efficient alternative for simple cases via the same plugin interface.
- **Signature connection** (elaborating on 3.10): **pyHanko** for the PDF-side PAdES signature creation/embedding/verification (all baseline profiles B-B/B-T/B-LT/B-LTA, PKCS#11 for hardware token/HSM) - handles the technical PDF work, while the actual trust service remains with the connected external QTSP.

---

2. Domain Model

2.1 Core Entities

Entity	Description
Document	Versioned file with metadata, object type, status, access rights - all versions remain permanently preserved and retrievable (see 2.1a); optionally with a deletion date/deadline and a marker for physical forced deletion (see 5.2a)
Folder	Hierarchical container, itself carrying metadata/object type, carries permission inheritance
Circulation folder	Process-related container with references to all documents belonging to a given case - no own file content, but a reference structure (see 2.3); can likewise be given an optional deletion date/deadline (see 5.2a)
Alternative representation	Alternative representation derived from an original document for accessibility/failure resilience (e.g. .txt from .docx, transcript from video, see 2.4)
Process-specific working copy	Independently stored copy of a document adapted for a specific case (e.g. redaction for file inspection), kept separate from plain versioning (see 2.3)
Electronic signature	Cryptographic proof (SES/AES/QES per eIDAS) for a specific document version, including validation status (see 3.10)
OCR text layer	Searchable text recognition for a scanned/image-based document, linked to the respective version (see 3.9)
Object type (custom object definition)	Defines attributes, required fields, validation rules for documents/ folders
Condition/ rule	Rule set that is checked on creation/modification (e.g. "attribute X must be set", "name must match pattern Y")

Entity	Description
(constraint)	
BPMN process	Executable or imported BPMN 2.0 model that guides documents/ circulation folders through processing steps (see 7.1)
Permission role	RBAC role, assignable per folder/object type
User/group	Local or synchronized from AD
Audit event	Immutable log entry for every relevant action
License	Defines the scope/limits of a tenant or an installation

2.1a Versioning (Clarification)

All versions of a document remain permanently preserved and retrievable over the entire retention period - no overwriting, no automatic discarding of older states (basic principle from 4.2/5.1). This applies unchanged to alternative representations (2.4) and process-specific working copies (2.3) as well: both are independent, likewise versioned objects, not transient intermediate states.

2.2 Object Types & Custom Constraints

Object types are defined as a schema (similar to the JSON Schema principle):

```
{
  "objectType": "Rechnung",
  "appliesTo": "document",
  "allowedParentTypes": ["Rechnungsordner", "Projektordner"],
  "attributes": [
    { "name": "Rechnungsnummer", "type": "string", "required": true, "pattern": "^RE-\\d{
    { "name": "Betrag", "type": "decimal", "required": true },
    { "name": "Kostenstelle", "type": "string", "required": false }
  ],
  "namingConstraints": {
    "mustContain": ["Rechnungsnummer"],
    "pattern": "{Rechnungsnummer}_{Datum}"
  },
  "conditions": [
    { "if": "Betrag > 10000", "then": "require:Kostenstelle" }
  ]
}
```

The rule engine (see 4.5) evaluates such conditions on creation, modification, and status transitions. The same principle applies to folder object types (e.g. "this folder type always requires a 'Project number' attribute"). `allowedParentTypes` is optional (if absent, the type is considered placeable anywhere, as before) - see 2.2a for the enforced hierarchy and 2.2b for the layout-related attributes (`icon` , form layout).

2.2a Enforced Object Hierarchy & Visual Classification

So that a filing structure desired by the operator can be enforced not merely by convention but **compulsorily**, every object type (both folder and document classes) can name one or more permitted parent folder classes via `allowedParentTypes` (2.2):

- **Multiple entries allowed** (OR combination): a document/folder class can name several permitted parent classes, e.g. because it can meaningfully occur under two different parent folder types from a business perspective.
- **Root as a valid target**: the special value `"$ROOT"` allows placement directly beneath the root (analogous to existing root special cases such as the `"root"` resource in the permission model, 4.1). A typical example from practice: `meinTopLevelOrd` allows only `"$ROOT"` , `meinSecondLevelOrd` allows only `meinTopLevelOrd` , `meinDoc` allows only

`meinSecondLevelOrd` - this produces an enforced, multi-level filing hierarchy instead of a freely navigable folder structure.

- **Enforcement**: the check is carried out by the same rule/constraint engine (4.5) that also evaluates the other object type conditions - both when creating *and* when moving a folder/document, it is checked whether the class of the target parent folder is contained in the `allowedParentTypes` of the class being placed. A violation is rejected like any other constraint violation (2.2), not merely displayed as a warning.
- **Export/importability**: `allowedParentTypes` is an ordinary field of the object type definition and is thus automatically part of the JSON export/import already provided for object types (7.3) - no separate configuration layer is needed.

Additionally, every **folder class** can carry an `icon` (2.2b) that is displayed in the User UI Explorer before the name of every instance of that class (8) - this allows folders of different classes to be distinguished at a glance, without having to read the full object type name.

2.2b Form Layouts (Display, Search, Upload)

In addition to its attributes (2.2), every object type carries a **form layout**: a JSON representation that describes how the attributes of this type are arranged and labeled in the User UI. The layout is not authored by hand as JSON/text, but **generated via a GUI editor in the Admin UI** (8): the administering person selects which attributes belong to the class, assigns a descriptive display name per attribute (independent of the internal technical name), and specifies which attributes are required fields - from this information the system automatically derives a sensible default layout ("smart layout"), which is stored server-side as JSON.

- **One layout, three purposes**: the same layout format controls (a) the metadata display/ editing of a document/folder of this class, (b) the filter fields of the search mask (3.7) for this object type, and (c) the input fields of the upload mask. Each of the three purposes can - via the same **Layout Designer** in the Admin UI - be adjusted to deviate from the generated default layout (e.g. an attribute that is required in the upload dialog but remains optional in the search mask), without changing the underlying object type definition itself.
- **Grid model**: a layout is organized into rows and/or columns - attributes can be grouped side by side (column-wise) or one below the other (row-wise), freely determined by the administering person in the Layout Designer.
- **Responsive behavior**: below a configurable width, a multi-column layout is always displayed as a single column (stacked row-wise only), regardless of its authored

configuration - legibility in narrow display areas takes precedence over the originally designed column split. What matters (since P23-S6, via CSS container queries) is the actual width of the form itself, not the window width - in the dockable workspace (8), a panel can be freely narrowed independently of the window, and precisely this case should stack, even if the surrounding browser window remains wide.
- Like the object type definition itself (2.2), the layout is also part of the admin-side configuration - end users of the User UI see only the result, not the editor.

2.3 Circulation Folders & Process-Specific Working Copies

Circulation folders

- A circulation folder bundles **references** (not own copies) to all documents belonging to a specific case - comparable to a digital file/routing folder that moves through various departments or instances via a BPMN process (7.1).
- A circulation folder is itself an independent, RBAC- and constraint-capable object (2.2) with its own object type, its own attributes (e.g. case number, responsible department, due date), and its own lifecycle via the Workflow Engine (7.1).
- **Reference behavior during active processing vs. on completion** (two-stage model):
  - **While the case is running**, the circulation folder references, for each contained document, **dynamically the respective most recent version** - if a referenced document is further modified during processing (new version, 2.1a), the circulation folder automatically points to this latest state, without the reference needing to be manually updated.
  - **On completion of the circulation folder** (reaching the final state in the associated BPMN process, 7.1), the reference structure is **fixed**: for every document contained at that point in time, the then-current version is committed as a **completion snapshot** - from this moment on, the circulation folder no longer dynamically points to "the respective latest version", but concretely to exactly the version states that existed at the time of completion. Later changes to the referenced original documents (insofar as they continue to be edited independently) no longer alter the fixed completion snapshot.
  - This completion snapshot is automatically archived together with the circulation folder (see archivability below) and is the basis for later being able to seamlessly trace the **actual processing state at case completion** - regardless of how the referenced documents subsequently develop.

- **Archivability**: circulation folders are subject to the same rules as documents - records disposal/long-term archiving (5.6) includes circulation folders together with their (post-completion fixed) reference structure, not just the referenced individual documents. The completion snapshot ensures that the referenced document versions remain unambiguously and permanently traceable at the time of case completion.
- When a referenced individual document is deleted/disposed of, the reference in the circulation folder remains traceably present (a note that it "existed, is now archived/ disposed of"), instead of disappearing without comment - important for later reconstructability of a case.

Process-specific working copies

- For cases in which a document is needed in an adapted form for the purposes of a specific case - a classic example: a **redaction** for a citizen's file inspection, because the original document contains third parties' personal data - this adapted version is **not treated as a new version of the original document**, but as an independent, separately stored working copy with a reference to the original document (including the specific starting version) and the triggering case/circulation folder.
- This separation is deliberate: the original document remains unchanged and fully viewable (for authorized persons), while the working copy is visible/usable exclusively for its respective case purpose - its own access rights, its own object type (e.g. "working copy - redaction"), its own retention rule (which can, for example, be tied to the duration of the triggering case rather than to the original's retention period).
- Creation, modification, and deletion of such working copies are fully audited like any other document action (5.3) and are likewise versioned (2.1a).

2.4 Alternative Representations (Accessibility & Failure Resilience)

In addition to the regular preview/rendering function (Rendering/Preview Service, 3.7), the system supports **alternative representations**: independently stored alternative representations derived from an original document, which serve two purposes:

- **Failure resilience/partial-outage access**: e.g. a .txt extraction from a .docx in case Word processing/preview is temporarily unavailable, or a .pdf conversion from a .pptx in case PowerPoint processing has problems - users thereby retain access to the content in an

alternative, more robust format even during a partial outage of individual rendering components.
- **Accessibility**: e.g. a transcription of a video for deaf users, or generally alternative text representations for screen reader accessibility.

Technically:

- Alternative representations are generated by the Rendering/Preview Service (3.7) (rule-based/configurable which target formats are generated per source format - e.g. "for .docx, always maintain a .txt alternative representation", "for video, always generate a transcript if a transcription plugin is available") and stored as independent objects linked to the original - not as a transient cache entry regenerated on demand, since availability should not depend on the functioning of the original renderer, particularly in a failure scenario.
- As with rendering in general, the actual generation is **plugin-/service-based** (3.3/3.8) - new target formats or new transcription/conversion capabilities (e.g. a new video transcription plugin) can be added following the "adding on" principle, without changing the core.
- Alternative representations are versioned like the original (2.1a) and are subject to the same permissions as the underlying document, unless configured differently.
- Accessibility alternative representations are conceptually clearly distinct from process-specific working copies (2.3): the former are a complete, faithful alternative representation of the same content for all users, the latter are a deliberately, content-altered (e.g. redacted) version for a specific case purpose.

2.5 Area Structure: Document Root and Special Areas

The actual file storage follows a structure that is freely configurable via object types (2.2) and the enforced hierarchy (2.2a) - a typical, but not fixed, example from administrative practice: a **file** (document object type) resides in a **file plan** (folder object type), which resides in a **department** (folder object type), which resides directly beneath the **document root** (`"$ROOT"`, 2.2a). Every installation can name these classes/levels however it likes, provide for more or fewer levels, or allow several parallel root classes - the example only serves to illustrate the mechanism from 2.2a.

In addition to this configurable document root, the system provides, at the same structural level, further **special areas** provided by the system itself - comparable to the special folders of an email program (inbox, trash, ...), but tailored functionally to records management. A special

area is not an ordinary, freely creatable folder: it exists exactly once per installation (or per tenant, 3a), its content arises predominantly from system operations rather than manual creation, and it carries its own access regime, configurable independently of the rest of the RBAC model (4.1).

Special area	Purpose	Access	Reference
Inbox/outbox	External correspondence (e.g. emails from/to other agencies, handovers from/to other installations) that is **not yet assigned to any case** lands here instead of directly in a file plan.	Only a dedicated **mail room role** sees/processes the unreviewed intake; only after review/ assignment (manually or via a BPMN intake step, 7.1) does an item move into the file plan or into a newly/existing assigned circulation folder (2.3).	3.3 (connector principle for the technical receipt, e.g. email connector), 7.1 (assignment workflow), 7.4 (a federated inbound/ outbound item from another installation is conceptually the same case)
Trash	Collection point for documents/ circulation folders marked for deletion by regular users (soft delete, 5.2) - no immediate deletion.	Regular users can move items in (mark them), but cannot permanently delete from there.	4.6, 5.2

Special area	Purpose	Access	Reference
		Permanent deletion is reserved for a separate, scoped-down admin role, **deletion administration** (example of a domain-separated admin role, 4.6).	
Classified documents trash	Functionally identical to the regular trash, but **structurally separate** for documents classified as classified documents (marked via an object type/ attribute, 2.2 - a dedicated classification-level system is not part of this concept).	Its own, narrower **deletion administration (classified documents)** role, not necessarily staffed by the same people as the regular deletion administration - separation so that regular deletion admin staff do not automatically gain insight into content classified as classified documents.	4.6
Personal trash	Person-specific view: shows each user exclusively the items **they themselves** marked for deletion,	Each user sees only their own entries - no	5.2

Special area	Purpose	Access	Reference
	together with status ("awaiting deletion administration"), independent of the complete trash inventory visible to deletion admins.	insight into other people's deletion markers.	
Quarantine	Documents that fail the mandatory virus scan before release (10.3) land here instead of in the regular inventory.	A dedicated, narrowly scoped role may view a quarantine case, permanently delete it, or - after clarifying a false positive - release it; the release is an audited action (5.3), not a silent restoration.	10.3
Contacts	Directory for finding other staff members - primarily within one's own installation (based on the existing user management, 4.4), **optionally cross-installation**, provided a federation (7.4) is configured and enabled for contact search. Deliberately distinct from the Federation Hub's address book (7.4), which lists *installations* as participants, not individual persons - a cross-installation contact search is a separate capability built on top of the Federation Hub, not an	Visibility configurable per installation (e.g. own department only, whole installation, federated installations).	4.4, 7.4

Special area	Purpose	Access	Reference
	automatic side effect of the existing federation function.		
Records disposal	Files/circulation folders already disposed of but still within the transition period (5.6) remain searchable/viewable here, instead of being reconstructable only via metadata/audit trail references. After the transition period expires, an item disappears from this area (regular deletion in the active system, 5.6), but remains traceable in the long-term archive/audit trail.	Access generally restricted to the same roles that had access before disposal, additionally possibly a dedicated archive/ registry role.	5.6, 7.2
Templates	Reusable structural templates, e.g. a complete file-plan skeleton (folder classes including their usual sub-levels) for a newly established department. Technically the same foundation as the JSON structure export/ import already present (7.3): a template is a named structure export stored in the templates area, applying it is a targeted structure import beneath a chosen target location.	Creating/ modifying templates is administratively protected (e.g. domain-separated admin role, 4.6); applying a template can be more broadly permitted (e.g. for anyone who is otherwise allowed to create a new department/file plan).	7.3
Team workspace	A **permanent** group work area for ongoing project/department	Access exclusively for invited	4.1, 8

Special area	Purpose	Access	Reference
("teamspace", update P12b-S1/user decision)	collaboration, deliberately distinct from the circulation folder (2.3) and not for sequential document routing - contains its own folders/documents, shared appointments, and contacts. Self-managed: any authorized person can create a new team workspace and invite members (individual users or groups) without any administrative pre-setup being necessary.	members; the creating person manages the member list but can delegate this management to further members. Access regime configurable independently of the rest of the RBAC model (4.1) and scoped to the respective workspace, analogous to the other special areas in this table.	

Further special areas can be added at any time via the same principle ("adding on", 1.) - an obvious candidate from classic records management that is deliberately **not** (yet) part of this concept version, but could be added without a structural break: a **follow-up reminder (Wiedervorlage)** (documents/cases that are automatically resubmitted for processing at a later, specific point in time - technically the same timer infrastructure as the deletion reminder, 5.2a, and the SLA time monitoring, 7.1, just used for a different business purpose).

Distinction from the narrow icon-based navigation bar in the User UI (8): the "tasks/inbox" item mentioned there refers to the **workflow task list** (open approvals/processing steps from 7.1) and is **not** identical to the inbox/outbox described here (external correspondence not yet assigned) - both terms sound similar but mean different things and are named clearly separately in the UI/terminology.

---

3. Architecture

3.1 Basic Principle: Shared Database, Separate Schemas

- A single physical database instance (e.g. PostgreSQL cluster) for simpler operations (backup/restore, scaling, monitoring in one place).
- Every microservice owns **exactly one schema of its own**, which it alone reads from and writes to. Other services query data exclusively via the API of the respective owner service, never by direct access to another schema.
- Schema migrations are part of the respective owner's service pipeline; no service alters another's schema.
- For cross-service evaluations (reporting, search), no direct joins are performed; instead, work is done via events/read models (see 3.4) or a dedicated reporting service with its own replicated view.
- Technical enforcement: a DB user per service with GRANT restricted exclusively to its own schema; this prevents accidental coupling even in the presence of bugs.

3.2 Registry Service – Heart of the Distribution

The Registry Service handles two tightly interwoven tasks:

a) Service discovery (control flow)

- Every service registers itself at startup with: service type, instance ID, version, supported capabilities, health endpoint, reachability address.
- Heartbeat/health check at a configurable interval; if a service fails, it is removed from the active routing table.
- Other services ask the registry which instance of a needed service type is currently reachable (basis for client-side load balancing or referral to an internal gateway).

b) License brokering

- The Registry Service itself performs no license checking, but queries the dedicated License Service and passes the result on to registering services.
- On registration, every service receives a response with a license status for its component: licensed, demo, unlicensed.
- The License Service publishes changes (e.g. license expiring, limit exceeded) as an event;

the registry updates the information, and affected services react (e.g. switching to demo mode) without a restart.
- Thus the license logic itself is also just a service - new license models can be introduced without touching the registry.

3.3 Connector Architecture (CMIS, WebDAV, Custom)

- Every connector is an independent service with a uniform connector interface (capability description: read, write, metadata, locking, versioning - supported differently depending on the target system).
- Registration analogous to all other services via the registry - including license checking (e.g. the CMIS connector as its own licensable component, see section 9).
- A connector SDK (reference implementation + interface definition) makes it possible to build further/customer-specific connectors without altering the core - this is the technical core of the "adding on" promise.
- Versioning of the connector protocol (e.g. via an API version in the registration handshake), so that old connectors do not break on core updates.

3.3a Office Integration (Native, Cross-Platform)

Update P12b-S1/user decision: originally envisaged only as a low-priority extension candidate (12.2), adopted into the core scope at the user's request - explicitly **not** limited to Microsoft Office, but from the outset also intended for LibreOffice/OpenOffice and thus for Linux/Mac, not only Windows.

Two separate, but equally treated, extensions instead of a Windows-centric solution

- **Microsoft Office**: implemented via **Office Add-ins (Office.js)**, not via the older VSTO/COM technology. Office.js add-ins are web-based (HTML/JS task pane + ribbon commands, executed in a sandboxed webview inside Office) and thereby run across systems wherever Microsoft Office itself runs (Windows, Mac, Web) - a COM/VSTO solution, by contrast, would be structurally confined to Windows, regardless of one's own development effort.
- **LibreOffice/OpenOffice**: implemented as a **UNO API extension** (`.oxt` extension package) - the same extension format works for both programs (OpenOffice/LibreOffice share the UNO base) and is genuinely cross-platform, since LibreOffice/OpenOffice themselves run on Windows, Linux, and Mac.

- Both extensions use, for the actual file transfer, the same already existing connector/API path (3.3, or a lean dedicated REST interface for add-in-specific actions) instead of their own, parallel document logic - consistent with the plugin/connector basic principle (3.3/3.8).

Targeted functional scope (in both extensions, insofar as the respective platform allows)

- Opening/saving a document directly from/to the DMS, without having to separately call up the DMS interface.
- Editing metadata/attributes of the open document inline in the task pane (2.2), instead of switching applications for that purpose.
- Starting a workflow (7.1) directly from the open document, or continuing/completing an open task on it.
- Access to a central, role-based library of templates/text building blocks when creating a new document from within the DMS.

Deliberate limitation: Microsoft Office itself does not exist natively for Linux - under Linux, therefore, only the LibreOffice/OpenOffice path covers this integration; an MS Office extension cannot take effect there. This is a limitation of the Office product itself, not a gap in this concept.

3.4 Event Bus for Auditability and Decoupling

In addition to the shared DB, an event bus is introduced:

- Every relevant action (document created, read, modified, deleted, permission changed, workflow step traversed, login, etc.) generates an event.
- The Audit Service consumes all events and writes them immutably (append-only, optionally with cryptographic chaining/hash chain for tamper-evidence) into its own schema.
- Other services (search index, notification service, reporting) can consume the same event stream without directly querying the Audit Service or other owner schemas - this reduces coupling even though the foundation is a shared DB.
- Benefit: auditability is structurally enforced by the architecture (no event = no action), not dependent on the discipline of individual developers.

Concrete technology choice: event bus as an interchangeable backend, analogous to the

storage abstraction (3.6)

- Rather than committing early to a single event bus technology, the event bus - consistent with the plugin/interchangeability principle of the rest of the system - is operated behind its own abstraction layer, so that the concrete technology can be chosen per installation.
- **Default setting: NATS JetStream.** This best fits the model of many fully isolated individual installations (3a): operation as a single, lightweight binary without the cluster/ operational overhead of a full-fledged Kafka setup, well suited for request-reply patterns between services in addition to pure event streaming, and resource-efficient enough to run per installation (instead of centrally shared).
- **Recommended alternative for large individual installations: Apache Kafka.** Sensible as soon as a single installation needs very high throughput, very long/permanent retention of the event stream (beyond the regular audit retention, as a technical replay log), or deeper ecosystem integration (e.g. Kafka Connect for downstream evaluation tools) - at the cost of higher operational effort (multiple broker nodes in production, more tuning effort).
- Both options are addressable via the same internal event publishing/consumption interface - services themselves only know “publish event X” or “consume events of type Y”, not the concrete bus technology underneath.

3.5 API Gateway / BFF Layer

- Central API gateway for authentication (token validation), rate limiting, routing to backend services (using the registry).
- One lean backend-for-frontend per web UI (user, admin, possibly further ones) that handles UI-specific aggregation, instead of letting the UI address many microservices directly.

3.6 Storage Abstraction Layer

The actual file content is not held in the relational shared DB (only metadata, references, checksums live there), but managed via a dedicated **Storage Service** that sits as an abstraction layer in front of different physical storage backends.

Backend connection as a plugin principle

- Analogous to the connector architecture (3.3), every storage backend (S3-compatible, NFS, Azure Blob, local filesystem, further ones in the future) is an interchangeable **storage**

backend plugin with a uniform interface (write, read, delete, existence check, checksum).
- New backends can be added following the same “adding on” principle, without touching the Storage Service itself.

Redundant/parallel storage

- Configurable per tenant/instance: one or several backends simultaneously as a **storage target set** (e.g. 2× S3 from different providers + 1× NFS).
- **Any number of like-kind backend instances**: the target set is not a fixed mapping of “one backend type = one target” - the same backend type can appear multiple times, independently configured, within the target set (e.g. 2× S3 against different buckets/ providers in addition to 1× NFS mount), each with its own credentials/mount parameters/ identity file (see storage device swap sensitivity below). The upper limit on the count is purely operational (a sensible configuration is up to the operator), not architecturally bounded.
- Write strategy configurable (per object type/folder area, not only globally):
  - **Synchronous with quorum**: a write is only considered successful once a configurable minimum number of targets has confirmed it (e.g. 2 of 3) - stronger consistency guarantee, higher write latency. **Default setting for areas with high protection needs/integrity requirements** (e.g. audit-proof archived or particularly sensitive documents) - here consistency takes precedence over speed.
  - **Synchronous to the primary target, asynchronous to secondary targets**: faster response time, with subsequent replication and tracking of outstanding copies (retry queue, alerting on a target's persistent failure). **Default setting for general working operations**, where response time/throughput has priority and a short time window until full redundancy is acceptable.
  - Both defaults are pure factory settings and can be overridden per object type/folder - the decision ultimately rests with the operator.
- Checksums (e.g. SHA-256) are captured per copy and reconciled against the reference value stored in the shared DB - the basis for automated integrity checks (regular fixity check across all copies).
- Read access: configurable target priority (e.g. “nearest/fastest target first”, automatic fallback to the next copy on unreachability).
- Removing/adding a backend from/to an existing target set is a dedicated, auditable operation (rebalancing: catching up missing copies at the new target before an old target is removed) - technically related to the migration process from 7.2.

Storage device swap sensitivity

A particularly easy-to-overlook risk with physical/mountable backends (NFS, local filesystem, possibly interchangeable storage media behind them): a storage device is accidentally swapped, reset, or exchanged for an empty/wrong target, without the Storage Service readily noticing - a silent fallback to a wrong medium would be fatal precisely for redundancy/integrity.

- Every backend instance managed by the Storage Service creates, on first use, an **index/identity file** at its root point containing a stable, generated **device ID**.
- On startup, the Storage Service checks for every configured backend whether its identity file is present and matches the last known device ID.
- **Default setting: refuse to start** as soon as a configured backend's identity file is missing or does not match - in this case, the Storage Service deliberately does not come up, instead of silently continuing to work with a wrong/empty medium.
- **Admin-configurable exception**: the Admin UI (8) allows configuring that a start is permitted even with a missing/deviating backend, **provided at least one other configured backend in the target set is verifiably unchanged** (identity file matches) - the service then starts in a deliberately degraded redundancy state instead of refusing entirely.
- In this degraded case, **background replication is mandatory**, not optional: the Storage Service automatically catches up the missing copies onto the new/reset target (the same re-replication infrastructure as in regular rebalancing above), until the configured redundancy (quorum/target-set requirement) is restored - only afterwards is the target set again considered fully healthy.
- This entire process (start refusal, admin override, degraded start, re-replication progress) is auditable like any other storage action (3.4/5.3) and visible as a status in the Admin UI.

3.7 Overview of Core Services (Excerpt)

Service	Responsibility
Registry Service	Service discovery, license brokering, sensor registry for monitoring
Plugin Orchestration Service	Distribution/scaling of plugins across the server landscape, time-profile-aware placement

Service	Responsibility
License Service	License management/checking
Auth Service	Authentication (local + AD), session/token
Permission Service	RBAC management, rights evaluation including inheritance, scope locks, emergency shutdown state
Document Service	Document CRUD, versioning, locking
Storage Service	Abstraction over storage backends (S3, NFS, ...), redundant storage, fixity checks
Folder Service	Folder structure, inheritance logic
Object-Type Service	Object definitions, attributes
Rule/Constraint Engine	Condition checking on creation/modification
Workflow Engine	BPMN 2.0 modeling/import, process execution, per-step SLA time monitoring
Case Service	Circulation folder management, case references to documents, archiving of circulation folders
Audit Service	Immutable event log
Search Service	Full-text index, faceted search
Rendering/Preview Service	Thumbnails, PDF/A conversion, watermarks, alternative representations (accessibility/failure resilience)
Virus Scan Service	Upload checking
OCR Service	Text recognition for scanned/image-based documents, feeds search and alternative representations
Signature Service	eIDAS-compliant electronic signature (SES/AES/QES), broker in front of external QTSPs
Notification Service	Email/in-app/webhook

Service	Responsibility
Migration/Transfer Service	Export/import of individual documents and entire structures
Config Import/Export Service	JSON configuration management
Delta/Comparison Service	Cross-installation configuration comparison (base instance vs. comparison instance), configurable scope, ignore regex for object names
Query & Trace Service	Central query/manipulation language over audit/reporting/ monitoring read models
Backup/Restore Orchestrator	Coordinated backup/restore across storage/DB/configuration, deletion reconciliation after restore, maintenance-mode coupling
Connector Services (CMIS, WebDAV, Custom, ...)	Connection to external repositories
Federation Hub Service	Cross-installation workflow brokering (address book + switching center, outside individual installations)

3.7a Full-Text Search: Query Language (Update P12b-S1/User Decision)

The full-text search should, in the functional scope of its query language, **match up with market-standard reference products**, not just offer simple keyword search:

- **Boolean combination with clear precedence rules**: AND/OR/NOT combinable, with a defined evaluation order instead of implicit/ambiguous combination of multiple search terms.
- **Phrase search** (exact word sequence in quotation marks) and **wildcards** (placeholders for partial words).
- **Fuzzy search** (tolerant match finding for typos/spelling variants, with an adjustable tolerance level) and **proximity search** (two terms count as a match if they occur within a configurable word distance of each other) - capabilities that go beyond pure Boolean/

phrase search and that a standard full-text search alone does not cover.
- Technically based on **Postgres full-text search** (Boolean/phrase/prefix search already native) supplemented with `pg_trgm` (trigram similarity) for the fuzzy component - deliberately no additional external search engine (Elasticsearch/Solr) as a mandatory dependency, consistent with the already-made fundamental decision to use Postgres as the sole data store (3.1).
- Saved/reusable searches remain distinct from free ad-hoc full-text search (faceted search/ filters, Search Service).

3.8 Plugin Orchestration & Automatic Load Distribution

The "adding on" principle (1., 3.2, 3.3) is extended with an active orchestration layer: new services/connectors/processing steps are not only passively detected (registry), but actively distributed across the available server landscape and, if needed, automatically scaled by a dedicated **Plugin Orchestration Service** - comparable to a lean, domain-adapted scheduler/ orchestrator (in the spirit of Kubernetes, but with DMS-specific placement logic on top of, or in combination with, the actual container platform).

Basic principle

- Every "addable" element (connector, rendering backend, rule plugin, storage backend adapter, migration worker, ...) is described as a **plugin package** with a manifest: required resources (CPU/RAM/IO profile), supported scaling type (stateless horizontally scalable vs. exactly one instance), dependencies on other services, and optionally a **load profile** (see below).
- Via the registry (3.2), the Plugin Orchestration Service knows the current state of all servers/nodes in the cluster (utilization, free capacity, currently running plugin instances) and decides on which node a new or additional instance is started.
- Administrators merely provide a plugin (upload/registration of the package) - placement, scaling, and, if applicable, migration to another node during load spikes happens automatically, without a manual assignment of "plugin X runs on server Y".

Time-profile-aware placement (core requirement)

- Much of the work in a DMS is not evenly distributed across the day (e.g. nightly fixity checks in the Storage Service, reporting aggregations, bulk migration jobs, AD synchronization outside core working hours, retention/deletion runs).

- Plugins can declare a **load profile** (e.g. "runs preferably at night", "runs periodically every X minutes", "runs interactively/on-demand with user access during the day") - the orchestrator uses this profile to preferably place temporally complementary jobs on the same node (e.g. an interactive service heavily loaded during the day and a batch job running at night share the same server, instead of permanently reserving separate capacity for both).
- Even without an explicitly declared load profile, the orchestrator observes the actual historical utilization per plugin type (a simple time-series evaluation based on the monitoring sensors, see 10.1) and derives an implicit profile from it over time - configuration is therefore optional, not mandatory.
- Redistribution of existing instances (rebalancing) happens in a controlled and gentle manner: running jobs are not hard-aborted; instead, new allocations take effect for new instances/jobs, and running processes are allowed to complete normally (drain behavior) before an instance is moved/terminated.

Placement algorithm: platform scheduler preferred, First Fit Decreasing as fallback

- If the installation runs on a platform with its own suitable scheduler (e.g. the Kubernetes/ OpenShift scheduler), the **Plugin Orchestration Service trusts that scheduler's placement decisions** and limits itself to the DMS-specific additional logic (time-profile assignment, license checking before placement, communication with the registry) - no parallel, competing placement decision alongside the already existing, mature platform scheduler.
- If no such platform scheduler is available (e.g. simple Docker/VM operation without an orchestration layer), the Plugin Orchestration Service takes over placement itself, as a **fallback using the First Fit Decreasing (FFD) method**: new/to-be-redistributed plugin instances are sorted in descending order by their resource requirements, and each instance is assigned to the first node on which it still finds sufficient free capacity - a proven, easily comprehensible bin-packing approach with good practical suitability, without rebuilding the complexity of a full-fledged scheduler of its own. The time-profile-aware grouping (see above) feeds in as an additional sorting/grouping criterion ahead of pure resource-size sorting.

Origin of the resource/utilization data

- Where the underlying platform itself already provides reliable resource/utilization data (e.g. the metrics server/scheduler of a Kubernetes/OpenShift environment), the **Plugin**

Orchestration Service preferentially relies on it, instead of redundantly collecting data itself - this avoids duplicate overhead and uses proven, platform-native mechanisms.
- Only when the platform does not provide this data, or not at sufficient granularity, does the orchestrator collect it via its own mechanism (at its core the same sensor infrastructure described in 10.1).
- **Cold-start placement** (new plugin with no historical utilization at all): the orchestrator first falls back on **static reference values that the plugin itself can provide in its manifest** (e.g. expected CPU/RAM demand per instance). If a plugin provides no such values, the orchestrator instead uses the **median of the actually observed resource usage of all currently registered plugins of the same type/category** as a provisional estimate, until enough of its own observation data is available for the new plugin and the estimate is replaced by real measurements.

Distinction from the registry

- The registry (3.2) answers the question "who is currently reachable and licensed" (control flow, reactive).
- The Plugin Orchestration Service answers the question "where should something run so that the server landscape as a whole is optimally utilized" (active control). Both services work closely together: the orchestrator reports placement decisions to the registry, so that other services can continue to correctly resolve the currently reachable instances.
- The Plugin Orchestration Service itself is - taken to its logical conclusion - also a service whose decisions are audited (which plugin was placed/moved when on which node) and whose activity is observable via the sensor/monitoring layer (10.1).

3.9 OCR Service (Text Recognition)

A fixed component of intake capture, not an optional add-on: scanned or image-based documents (e.g. scanned paper mail, faxes, image PDFs without a text layer) automatically go through text recognition so that they become searchable and can be further processed by content (e.g. for the constraint engine, 2.2, or the query console, 6.1).

- **Pipeline placement**: upload → virus scan → **OCR (if the document contains no usable text layer)** → rendering/preview (3.7) → search indexing (Search Service). Whether a document needs OCR is detected automatically based on the existing text layer, not enforced blanket-wide for every file type.
- **Plugin-based** (3.3/3.8): the concrete OCR engine is interchangeable, analogous to storage

backends and connectors - new/better engines can be added later without changing the core.

- **Concrete recommendation** (see 1a): **PaddleOCR** as the default engine (Apache 2.0 licensed, fits the open-source aspiration) - considerably stronger than classic Tesseract on complex layouts, tables, and multilingual documents, which is relevant for administrative/business documents (forms, tables, mixed layouts). **Tesseract** remains selectable via the same plugin interface as a resource-efficient alternative for simple cases or particularly constrained environments (lower resource demand, no GPU needed).
- The result (OCR text layer) is **linked to the respective document version and stored permanently** (not recomputed on every search) and directly feeds the Search Service (3.7).
- The generated searchable text layer can also serve as the basis for a text-based alternative representation (2.4) (e.g. a plain .txt extraction of a scanned document) - the same infrastructure is used for two purposes: search and accessibility/failure resilience.
- **Configurability (no always-on requirement)**: although, per the paragraph above, OCR is a fixed component of intake capture, operation of the component itself remains configurable, rather than being mandatorily provisioned in every installation:
  - **Fully disableable** (`ocrEnabled: false`): if OCR is not desired/licensed/needed for an installation (e.g. because only already text-based documents come in), the OCR Service itself **does not need to be deployed** - consistent with the "adding on" principle (1.): the absence of a plugin is a regular state, not an error case. Without an active OCR Service, the corresponding documents remain without a text layer (search/alternative representation then fall back only on whatever native text may exist).
  - **Maximum word cap**: a configurable threshold above which a document (estimated by page/image size) is skipped as "too large for OCR", instead of investing unlimited compute time in very large scans - a cost/performance safety valve, not a functional requirement.
  - **Processing batch size**: configurable, how many pages/documents are processed together per engine call/worker cycle - a lever for throughput vs. resource consumption (memory demand per batch), sensible differently depending on installation size.
  - All three settings are Admin UI configuration (8), not a code/deployment change.
- OCR results are audited like any other document processing (5.3); for particularly error-prone/uncertain recognitions (low confidence value of the engine), a manual review can optionally be triggered as a BPMN process step (7.1), instead of passing through faulty OCR results without comment.

3.10 Signature Connection (eIDAS-Compliant Electronic Signature)

Electronic signature is supported as a fixed component of approval processes - technically as a **broker in front of external, certified trust service providers (Qualified Trust Service Providers, QTSP)**, not as its own trust service: the legal recognition of a qualified electronic signature requires its own accreditation under eIDAS, which no software product can "bring along" itself - the system instead integrates already-accredited providers, similar to the other connector architectures (3.3).

- **Signature Service** as a dedicated core service that communicates with external QTSPs via interchangeable **signature provider connectors** (plugin principle as with CMIS/WebDAV, 3.3) (e.g. for qualified signatures), or - for lower trust levels - can also work purely internally with the system's own keys.
- **Supported signature levels per eIDAS**, configurable per object type/process step as to which minimum level is required:
  - **SES (Simple Electronic Signature)**: simple electronic confirmation, generatable internally by the system, no external provider needed.
  - **AES (Advanced Electronic Signature)**: uniquely attributable to a person, typically via an external provider or a certificate.
  - **QES (Qualified Electronic Signature)**: highest level, mandatorily via an accredited QTSP, legally equivalent to a handwritten signature.
- **Format**: PAdES for PDF documents (standard case), XAdES/CAdES for other formats, chosen automatically depending on the document type.
- **Integration into the BPMN processes (7.1)**: a new task type, **"Signature Task"**, supplements Manual/Automatic Tasks - a process step can require a signature from an authorized person before the process continues; the signature is bound to the specific document version (2.1a).
- **Verification**: signatures can be checked for validity at any time (certificate chain, revocation status, timestamp) - important particularly for long-term archiving/records disposal (5.6), where the validity of a signature must remain provable even years later (the PAdES-B-LTA profile with a continuous timestamp chain covers exactly this case).
- **Concrete library recommendation** (see 1a): **pyHanko** for the PDF-side generation/ embedding/verification of PAdES signatures (supports all common PAdES baseline profiles B-B/B-T/B-LT/B-LTA as well as PKCS#11 for hardware token/HSM integration) - handles the technical PDF signature work, while the actual trust service (certificate issuance, qualified

signature creation) remains with the connected external QTSP.
- Signed documents/their signature status are part of audit-proofness (5.1): a subsequent alteration of the signed content is detectable through the cryptographic signature itself, in addition to the audit trail that already exists anyway (5.3).

3a. Multiple Installations (Installation ≠ Tenant)

Decision: **no multi-tenant architecture within a single installation**; instead, a customer can operate **multiple fully independent installations** of the same system (e.g. an IT service provider who provisions a separate system instance each for two of its end customers).

- Every installation is fully isolated: its own shared DB instance (or at least its own set of schemas), its own event bus, its own registry instance, its own storage target set. No shared data access between installations.
- This avoids the complexity and risks of tenant separation *within* one instance (data leaks between tenants, more complex rights checking) and fits the "adding on" principle: a new installation is, at its core, another complete set of services being started up.
- **Central management layer (optional, separate building block):** an overarching **fleet/ license management service** (independent of the individual installations) enables an operator to get an overview of multiple installations - in particular license assignment/ renewal (see 9.2), a basic health overview, **and central provisioning of new installations** (providing a new installation from a configuration template, triggered via this central layer instead of manually per installation). However, this service has **no** access to the document content of individual installations, only to their license/status reports.
- **Flexible deployment topology for the License Service**: both of the following operating modes are supported, without anything changing at the core of the License Service:
  - **Fully isolated/standalone**: an installation operates its own License Service, e.g. in its own, sealed-off namespace of a container platform (such as OpenShift/OCP) - no dependency on an external instance, maximum isolation.
  - **Central License Service for multiple installations**: a License Service runs in a central namespace/cluster area, and multiple installations (in their own namespaces) call this central service for their license checking, instead of each running their own - fitting the fleet management scenario, reducing operational effort with many installations.
  - Which model applies is a pure configuration/deployment decision of the registry (3.2)

when the License Service registers - the rest of the system technically notices no difference.

- Configuration export/import (7.3) is the intended way to set up a new installation from an existing one's configuration state (cloning), not a shared data pool.
- Every installation registers with the License Service (possibly with the overarching fleet management) using its own installation ID, against which its license is checked.

Fleet update orchestration (P12b-S1 addition)

The uninterrupted rolling updates from 10.5 solve the update problem **within a single installation** (a service is drained, a new instance takes over). An operator with many installations (reseller scenario, 3a) additionally needs a layer that **coordinates and staggers updates across many installations** - without this layer, the operator would be left updating each installation individually and manually. Market comparison (P12b-S1) confirms: none of the compared systems offers a built-in tool for this (see 12.1) - operators have to build it themselves. This gap is deliberately closed here, as an extension of the fleet/license management service already provided for:

- **Staggered rollout via waves/groups**: installations are grouped into named groups (e.g. "test installations", "wave 1", "wave 2"); a wave must be explicitly started, and the next wave only starts after the previous one is confirmed - no automatic chain reaction across the entire fleet at once. Individual installations can be specifically included/excluded (`include` / `exclude`), without changing the plan for the remaining installations.
- **Declarative update plan per installation**: a structured sequence of named steps (analogous to the phases from 7.2: preparation/locking, update, verification, release) instead of an implicit "just update" - the plan is versioned and part of the configuration (7.3), not code.
- **Structured error decision instead of binary success/failure**: every step returns one of five decisions, so that a failed step does not end up categorized as a blanket "error", but can be handled in a targeted way:
  - `retry_later` - a temporary state, an automatic retry within a budget is safe (e.g. target installation briefly unreachable).
  - `wait_external` - an externally triggered operation is still running (e.g. the target installation is itself currently executing a rolling update step, 10.5) - no double start, just keep observing.
  - `manual_required` - an automatic decision would be risky or a business-judgment

call (e.g. a schema contract step that affects existing data) - an operator must actively approve; for this, the already existing generic four-eyes/approval principle (4.3) is reused, not reinvented.
    - `recoverable_failed` - a specific, named error exists; the operator can specifically retry, continue, or skip.
    - `fatal_contract` - the plan, script, or configuration is factually wrong, a retry would be pure waste of time, must be corrected before any further attempt.
- **Safe update windows per installation**: before risky steps (e.g. schema contract, see 10.5), the orchestration specifically activates a scope lock (4.7) or the system-wide maintenance mode (4.8) of the affected installation, rather than assuming that no one writes anyway during that time - after the step, the lock is automatically lifted again.
- **Backup before risky actions**: before every step that discards or overwrites existing state (e.g. a deliberate restart of the update plan), a backup is automatically created (reusing the existing backup mechanism, 10.4) - a restore in this context verifies that the plan name and plan fingerprint match the currently loaded plan, so that no backup belonging to a different run is accidentally applied.
- **Fully audited** (5.3) and observable via the existing monitoring (10.1) - error class, current step, and progress per installation are regular sensor/dashboard values, not a separate observation tool.
- This layer does not replace 10.5 (which remains the basis for the actual instance switch *within* an installation), but sets up a coordination layer *above* it, which specifically triggers 10.5 per installation and tracks overall progress across many installations.

---

4. Permission Model

4.1 RBAC as the Foundation

- Roles are defined centrally in the Permission Service and can be assigned per object type as well as per folder.
- **Inheritance**: permissions are inherited downward from parent folders, with an explicit override option at deeper levels (standard DMS behavior, e.g. as with SharePoint/Alfresco).
- For performance with deep hierarchies: effective permissions are not recomputed across the entire chain on every request, but held in a materialized, event-driven, updated cache

(invalidation via events from 3.4 when permissions or structure change).
- Role assignment both to individual users and to groups (local or AD groups, see 4.4).

4.2 Document Lock During Editing (Locking)

- If a user opens a document from within an application (e.g. Word via WebDAV/CMIS connection) for editing, an **editing lock** is automatically set that prevents other write access (read access remains possible depending on configuration).
- The lock is managed by the Document Service and is bound to the user plus session.
- **Release mechanisms:**
  - regularly by the user on completing editing (check-in),
  - automatically after a configurable timeout without activity (e.g. session/connection dropped),
  - **administrative force-unlock**: a role with the corresponding permission can manually lift a lock (e.g. for staff absent for an extended period). This action is itself a particularly sensitive audit case (see 4.3) and should optionally be subject to the four-eyes principle.
    Conflict handling on force-unlock (elaborated):

Goal: the original editor must never silently lose work, even if the lock was administratively lifted.

1. On force-unlock, the lock is not immediately removed entirely, but placed into a state of **"lifted, but monitored"**: the document becomes editable again for other users, but the original lock holder is informed via notification that their lock was administratively lifted (including the reason, if recorded).
2. If, in the meantime, another user uploads a new version (regular check-in), it is adopted as the new main version - this process is part of normal versioning.
3. If the **original** editor subsequently also attempts to upload a state (e.g. because they were unaware of the release or their client still had the file open), the Document Service detects, from the version chain, that their starting state is no longer the current version. Their upload is **not** applied as an overwrite, but stored as a **conflict copy** (e.g. "Document_conflict__") alongside the current version.
4. Both states (current main version + conflict copy) are visibly linked in the audit trail and in the UI; a manual merge by the parties involved or by an authorized person is possible via regular version management.

5. The entire process (force-unlock, notification, any resulting conflict copy) is fully audited and, as described in 4.3, can optionally be subject to the four-eyes principle.

This means a force-unlock never causes an edit to be lost without visibility; in the worst case, a visible conflict copy results, instead of silent data loss.

Deliberate limitation: no real-time simultaneous editing (Update P12b-S1) - the locking model is deliberately **exclusive** (one editor at a time), not real-time co-authoring by multiple people simultaneously on the same document (as, for example, cloud office suites offer via their own operational-transform/CRDT infrastructure). Such real-time collaboration would require a dedicated, very costly editor infrastructure that is out of proportion to a DMS's core purpose - comparable market products generally do not build this themselves either, but instead hand off to an external cloud-office integration for that purpose. The exclusive locking model with clear conflict-copy handling (above) remains the deliberately chosen, considerably simpler path.

4.2a Public Share Link (Update P12b-S1/User Decision)

- For a single document, an authorized person can generate a **time-limited, unauthenticated share link** - e.g. to pass a document to an external person without their own access, without having to send the document separately by email.
- **Globally switchable off, not only deniable per user/role**: a system switch allows the function to be completely disabled for the entire installation - in this case, the function is **not offered in the UI at all** (no menu item, no reachable API route), instead of merely being hidden or blocked server-side. The default setting is deliberately left configurable, not a fixed system default (analogous to the already established pattern for the forced-deletion base setting, 5.2a).
- A share link **mandatorily requires an expiration date** (no permanently valid link) - the maximum validity period is configurable installation-wide.
- The link grants **exclusively read access to precisely this one document** (not to the surrounding folder, not to metadata beyond the minimum necessary for display) - no implicit access to other RBAC-protected content.
- Creation, access, and expiry of a share link are fully audited (5.3); a link can be prematurely revoked at any time by the creating person or an authorized admin role.
- Technically one of the few deliberate exceptions to this concept's otherwise consistently authenticated access model - the accessed page itself is one of the publicly reachable,

non-personalized exception pages already provided for in 8 (no full CSR access to the authenticated application).

4.3 Four-Eyes Principle (Configurable)

- Configurable per action type, not globally enforced. Examples of action types that can optionally require a four-eyes principle:
  - Release of a document/workflow step
  - Deletion (permanent, after the retention period)
  - Change of permissions/roles
  - Force-unlock of a document
  - Change of object type definitions or constraints
  - Import/export of configuration
  - Start of a migration/transfer operation
- Technically implemented as a generic **approval mechanism** in the Permission Service/ Workflow Engine: an action with an active four-eyes flag generates an approval request that must be confirmed by a second authorized person (not identical to the initiator) before it takes effect.
- Configuration is done per tenant/instance and is part of the JSON configuration export (see 7).

4.4 User Management: AD + Local

- The Auth Service supports in parallel:
  - **LDAP/Active Directory integration** (user and group synchronization, login via AD credentials/Kerberos/SSO optional),
  - **local accounts** (for external users, service accounts, systems without AD integration).
- Group memberships from AD are mapped to internal roles. This mapping is **fully configurable** and not hard-coded:
  - Administrable via the Admin UI and part of the JSON configuration export (7.3), so that a mapping can be transferred between installations.
  - Supports simple 1:1 mapping (AD group → internal role) as well as more complex rules (e.g. AD group X **and** attribute Y → role Z; multiple AD groups → one shared role).
  - Changes to the mapping only take effect after explicit approval/saving (no live editing with immediate, uncontrolled wide-reaching effect) and are themselves audited.

    - For unmapped AD groups: configurable default behavior (no role assigned vs. a defined default role).
- Synchronization interval configurable; deactivation/deletion of AD users must promptly affect access rights.

Concrete technology choice: OpenID Connect (OIDC) based on Keycloak

- **OpenID Connect** (built on OAuth2) is used consistently as the protocol: the Auth Service obtains/validates JWT access tokens, which enable stateless rights checking by every individual microservice (fitting the guiding principle of consistent statelessness, 1.) instead of requiring a central session lookup on every call.
- **Keycloak** is used as the concrete identity provider behind it, instead of developing AD/LDAP federation, multi-factor authentication, and session management in-house: Keycloak already fully covers OIDC, SAML 2.0, LDAP/AD federation, and identity brokering, is licensed under Apache 2.0 (fitting the open-source aspiration), and also uses PostgreSQL (3.1) for production operation - no additional database system needed.
- The concept's Auth Service thereby becomes a **lean adapter/broker in front of Keycloak** that translates the results (token, group memberships) into the internal RBAC model (4.1), instead of implementing its own IAM logic - considerably reduces development effort and security risk compared to an in-house build.
- **SAML 2.0** is available on top of this for installations with older ADFS/SAML-based federation requirements, likewise via Keycloak, without additional in-house development.

4.4a Delegation (Substitution During Absence, Update P12b-S1/User Decision)

- A person can designate one or more other people as a **delegate** (e.g. for the period of an absence) - the delegate can thereby take over tasks/approvals of the represented person on their behalf (7.1), without their credentials needing to be shared.
- **Deliberately implemented as an independent, clearly attributable action instead of an identity switch**: an action performed on the delegate's behalf is logged **under the delegate's own account**, with an additional note as to on whose behalf it was carried out (5.3) - no "logging in as" another person, which would dilute attributability in the audit trail.
- The scope of the delegation is configurable: full takeover of all open tasks of the represented person, or restricted to specific object types/processes/folder areas.

- A delegation has a start and end date (time-limited) and can be ended early at any time by the represented person or an authorized admin role.
- To be distinguished from full account takeover by support/admin staff (that remains a force-unlock-like, separately audited administrative action, 4.2) - the delegation is a regular business feature set up by the affected person themselves, not an administrative intervention.

4.5 Custom Bedingungen (Constraint Engine)

- Regel-Engine wertet bei Erstellung/Änderung von Dokumenten/Ordnern die im Objekttyp hinterlegten Bedingungen aus (siehe Beispiel in 2.2).
- Unterstützt mindestens: Pflichtfelder, bedingte Pflichtfelder (abhängig von anderen Attributwerten), Musterprüfung für Namen/Werte (Regex), Wertebereiche, Verweise auf andere Objekte.
- Sollte erweiterbar sein um eigene Validierungs-Plugins (ähnlich der Connector-Idee: neue Regeltypen als "dazustellbare" Erweiterung).

4.6 Superuser-Prinzip (Break-Glass) & domänengetrennte Admin-Rollen

Ziel: Die Angriffsfläche eines kompromittierten administrativen Kontos minimieren, indem es im Normalbetrieb gar keinen dauerhaft nutzbaren Vollzugriffs-Account gibt.

Superuser als deaktivierter Break-Glass-Zugang

- Nach der Ersteinrichtung wird der Superuser-Zugang (Zugriff auf wirklich alle Funktionen ohne Einschränkung, inkl. Manipulationsmodus der Query-Konsole aus 6.1) standardmäßig **deaktiviert**.
- Reaktivierung erfordert **zwingend das Vier-Augen-Prinzip**, wobei die Berechtigung zur Freigabe nicht an zwei fest benannte Einzelkonten gebunden ist, sondern über **Gruppenzugehörigkeit** definiert wird: Zwei unterschiedliche Mitglieder einer (oder mehrerer) dafür vorgesehenen Berechtigungsgruppe(n) müssen die Aktivierung gemeinsam bestätigen. Die Gruppe sollte im Idealfall mehr als zwei Mitglieder umfassen, damit die Vier-Augen-Regel auch bei Abwesenheit Einzelner hart durchsetzbar bleibt, statt bei Ausfall einer der beiden fest benannten Personen zum Single Point of Failure zu werden.

- Die Aktivierung ist zeitlich befristet und deaktiviert sich **zusätzlich automatisch bei Inaktivität** (konfigurierbares Zeitfenster, **Standardwert: 10 Minuten** ohne Aktivität als Superuser) - unabhängig von einem eventuell separat konfigurierten maximalen Zeitfenster der Aktivierung insgesamt. Damit bleibt ein vergessenes, aber ungenutztes Superuser-Fenster nicht unnötig lange offen.
- Jede Aktivierung, jeder Login und jede im aktivierten Zustand durchgeführte Aktion wird mit erhöhter Priorität auditiert und kann optional zusätzlich eine sofortige Benachrichtigung an eine definierte Sicherheitsverantwortliche/einen Sicherheitsverantwortlichen auslösen.

Abgespeckte, domänengetrennte Admin-Rollen für den Alltag

- Für den laufenden Administrationsbetrieb existieren stattdessen von Anfang an mehrere **vordefinierte, im Funktionsumfang eingeschränkte Admin-Rollen**, die jeweils nur eine fachliche Domäne abdecken - z. B. getrennte Rollen für Nutzer-/Rechteverwaltung, für Objekttyp-/Workflow-Konfiguration, für Storage-/Backend-Verwaltung, für Lizenzverwaltung, für die Query-Konsole (6.1, i. d. R. nur lesend), sowie **Löschadministration** und, davon personell trennbar, **Löschadministration (Verschlusssachen)** für die endgültige Löschung aus den beiden getrennten Papierkorb-Bereichen (2.5).
- Diese Rollen sind **standardmäßig mitgeliefert** (keine Pflicht, sie erst selbst zu modellieren) und lassen sich wie jede andere RBAC-Rolle weiter anpassen/einschränken.
- Wird ein einzelnes dieser abgespeckten Admin-Konten kompromittiert, ist der mögliche Schaden strukturell auf dessen Domäne begrenzt - insbesondere kein Zugriff auf die Query-Konsolen-Manipulationsfunktion (6.1) oder auf domänenfremde Bereiche.
- Zusammenspiel mit 4.3 (Vier-Augen-Prinzip): einzelne Aktionen auch dieser abgespeckten Rollen können weiterhin zusätzlich dem Vier-Augen-Prinzip unterliegen, wo das konfiguriert ist - die Domänentrennung und das Vier-Augen-Prinzip sind unabhängige, sich ergänzende Schutzmechanismen.

4.7 Bereichssperren (Aussperren regulärer Nutzer aus Ordnerbereichen)

Ergänzend zum Locking einzelner Dokumente (4.2) kann ein **ganzer Ordnerbereich** (inkl. aller Unterordner) gezielt für reguläre Nutzer gesperrt werden, sodass dort - unabhängig von den sonst geltenden RBAC-Rechten - vorübergehend keine Schreib- (konfigurierbar auch: keine

Lese-)Zugriffe mehr möglich sind.

- Anwendungsfälle: bevorstehende Migration/Aussonderung eines Bereichs (7.2/5.6), Verdacht auf Datenintegritätsprobleme in einem Teilbaum, organisatorische Sperrungen (z. B. während einer Revision/Prüfung eines Aktenbereichs).
- Eine Bereichssperre ist eine eigene, klar von der regulären Rechtevergabe unterschiedene Konstrukt: sie **überlagert** RBAC-Rechte temporär, statt sie zu verändern - nach Aufhebung der Sperre gelten unmittelbar wieder die ursprünglichen Rechte, ohne dass diese vorher manuell entzogen und wieder vergeben werden mussten.
- Wer eine Bereichssperre setzen/aufheben darf, ist selbst RBAC-gesteuert (typischerweise eine der abgespeckten Admin-Rollen aus 4.6, ggf. mit Vier-Augen-Pflicht analog 4.3) und wird auditiert (5.3).
- Betroffene Nutzer erhalten beim Zugriffsversuch eine klare Rückmeldung samt - falls hinterlegt - Grund und voraussichtlicher Dauer der Sperre, statt eines unspezifischen Berechtigungsfehlers.

4.8 Not-Shutdown (systemweite Notfallsperre & Wartungsmodus)

Ein System weites, kurzfristig auslösbares Sperr-Werkzeug für den Fall eines Verdachts auf unautorisierten Zugriff - Ziel ist die schnellstmögliche Schadensbegrenzung, notfalls auf Kosten der Verfügbarkeit.

- **Auslösung**: Welche Rolle(n)/Gruppe(n) einen Not-Shutdown auslösen dürfen, ist **frei konfigurierbar** je Installation (keine fest im System verdrahtete Rolle) - typischerweise, aber nicht zwingend, eine kleine, eigens dafür vorgesehene Sicherheits-/Notfallrolle. Ob die Auslösung selbst zusätzlich dem **Vier-Augen-Prinzip** unterliegt, ist ebenfalls **optional konfigurierbar** (analog 4.3): Manche Installationen werden hier bewusst auf eine zweite Freigabe verzichten wollen, damit im akuten Verdachtsfall keine Verzögerung entsteht; andere - z. B. um Missbrauch der Notfallfunktion selbst vorzubeugen - werden auch hier eine zweite Bestätigung verlangen. Die Auslösung erfolgt über die Admin-UI, das CLI-Tool (6.2) oder einen dedizierten Notfall-Endpunkt.
- **Wirkung, sofort und systemweit**:
  - Alle regulären Nutzersitzungen werden beendet bzw. eingefroren; neue Logins außer für den Superuser werden abgelehnt.
  - Alle laufenden Workflow-Instanzen, geplanten/periodischen Jobs (inkl. Migrations-

/Aussonderungs-/AD-Sync-Läufe) und **alle Plugin-Instanzen** (3.8) werden angehalten - kontrolliert dort, wo ein sauberer Stopp ohne Datenverlust möglich ist, sonst hart, mit anschließender Konsistenzprüfung beim Wiederanlauf.
    - Schreibender Zugriff auf Storage (3.6) und Shared DB wird für alle Services außer den für die Notfallreaktion selbst nötigen unterbunden.
    - Föderierte Vorgänge über den Federation Hub (7.4) werden pausiert, nicht abgebrochen - offene Übergaben bleiben im Wartestand, statt inkonsistent abzureißen.
- **Wartungsmodus als Folgezustand**: Nach Auslösung befindet sich das System in einem expliziten Wartungszustand, der von jedem regulären UI/CLI-Zugriff klar signalisiert wird. Aus diesem Zustand heraus ist **ausschließlich der aktivierte Superuser** (4.6 - die Notfallsperre erzwingt/erlaubt hierfür die reguläre Break-Glass-Aktivierung) handlungsfähig, um die Lage zu prüfen und die Sperre gezielt wieder aufzuheben.
- Der gesamte Vorgang (Auslösung, Zeitpunkt, auslösende Person/Gruppe, spätere Aufhebung) wird mit höchster Priorität auditiert (5.3) und kann automatisch eine Alarmierung an hinterlegte Sicherheitsverantwortliche auslösen.
- Abgrenzung zur Bereichssperre (4.7): Der Not-Shutdown ist systemweit und für den akuten Sicherheitsvorfall gedacht, die Bereichssperre ist gezielt/organisatorisch und betrifft einen begrenzten Ordnerbereich im laufenden Regelbetrieb - beide Mechanismen sind unabhängig voneinander nutzbar.

---

5. Compliance & Auditierbarkeit

5.1 Revisionssicherheit

- Orientierung an **GoBD**-Grundsätzen: Nachvollziehbarkeit, Vollständigkeit, Richtigkeit, zeitgerechte Erfassung, Ordnung, Unveränderbarkeit.
- Als fachliche Referenzarchitektur bietet sich die BSI-Richtlinie **TR-ESOR** an (vertrauenswürdige Langzeitspeicherung), auch wenn keine 1:1-Umsetzung nötig ist.
- Einmal archivierte/freigegebene Dokumentversionen sind unveränderlich (WORM-artiges Verhalten auf Storage-Ebene, z. B. Objektspeicher mit Object Lock/Retention). **Einzige bewusste Ausnahme**: die verpflichtende, physische Zwangslöschung nach Frist (5.2a), wo eine gesetzliche Löschpflicht das Unveränderlichkeitsprinzip im Einzelfall gezielt und

nachvollziehbar überschreibt.

5.2 Aufbewahrung, Löschung, Legal Hold

- Aufbewahrungsfristen sind pro Objekttyp/Ordner konfigurierbar.
- **Legal Hold**: Möglichkeit, ein Dokument/eine Struktur unabhängig von der regulären Frist zu sperren (z. B. bei laufendem Rechtsstreit) - überschreibt die reguläre Löschfrist, bis der Hold aufgehoben wird.
- **DSGVO-Spannungsfeld**: Löschpflicht bei personenbezogenen Daten vs. Aufbewahrungspflicht. Lösungsansatz: Pseudonymisierung/Schwärzung einzelner personenbezogener Attribute statt Hardlöschung des gesamten Dokuments, wo eine Aufbewahrungspflicht besteht; echte Löschung nur, wo keine gesetzliche Pflicht entgegensteht.
- Papierkorb/Soft-Delete mit konfigurierbarer Wiederherstellungsfrist vor endgültiger Löschung.

5.2a Verpflichtende Löschung nach Frist (Zwangslöschung)

Ergänzend zu Aufbewahrungsfristen (die primär regeln, wie *lange* etwas mindestens erhalten bleiben muss) unterstützt das System das umgekehrte, ebenso wichtige Szenario: Dokumente oder Umlaufmappen, die aus Datenschutz- oder anderen rechtlichen Gründen nach Ablauf einer Frist **tatsächlich gelöscht werden müssen** - vom System durchsetzbar, nicht nur als organisatorischer Hinweis.

Konfiguration je Dokument/Umlaufmappe

- **Löschdatum bzw. "Löschen nach Zeitraum"**: ein konkretes Datum oder eine relative Frist (z. B. "5 Jahre nach Erstellung", "1 Jahr nach Vorgangsabschluss") kann je Dokument oder Umlaufmappe hinterlegt werden - entweder manuell gesetzt oder automatisch aus dem Objekttyp (2.2) übernommen, wenn der Objekttyp eine Standardfrist definiert.
- **Checkbox "vollständig löschen" (physische Löschung)**: Standardverhalten ohne diese Markierung ist die reguläre, bereits bestehende Soft-Delete-/Archivierungslogik (5.2/5.6) - das Objekt verschwindet aus der aktiven Nutzung, bleibt aber im Sinne der Revisionssicherheit (5.1) grundsätzlich erhalten. Ist die Checkbox gesetzt, wird stattdessen eine **echte, physische Löschung** durchgeführt: Das Objekt (inkl. aller Versionen, 2.1a, und aller redundanten Kopien über sämtliche konfigurierten Storage-Backends, 3.6) wird unwiederbringlich entfernt.

    - Das ist die eine bewusste, eng kontrollierte **Ausnahme vom sonst geltenden Unveränderlichkeits-/Revisionssicherheits-Prinzip** (5.1) - gerechtfertigt dadurch, dass hier eine gesetzliche Löschpflicht (statt einer Aufbewahrungspflicht) den Vorrang hat.
    - Bekannte technische Grenze, die transparent zu kommunizieren ist: Bereits bestehende **Backups**, die vor dem Löschzeitpunkt erstellt wurden, enthalten das Objekt ggf. noch bis zu ihrem eigenen, separat konfigurierten Ablauf/ihrer eigenen Rotation - vollständige Löschung im produktiven System bedeutet nicht automatisch sofortige Löschung aus jedem historischen Backup-Stand. Das ist eine allgemeine Grenze jeder Lösch-Funktionalität, keine Besonderheit dieses Konzepts, sollte aber im Betriebskonzept (Backup-Rotation) explizit berücksichtigt werden.
    - **Wichtige technische Voraussetzung auf Storage-Ebene** (Zusammenspiel mit 3.6/5.1): Wird das WORM-Verhalten aus 5.1 technisch über objektspeicherseitiges Object Lock umgesetzt, muss dafür bewusst der **"Governance"-Modus statt des "Compliance"-Modus** gewählt werden - Compliance-Mode-gesperrte Objekte lassen sich technisch von niemandem, auch nicht mit erweiterten Rechten, vor Fristablauf löschen, was die hier beschriebene Zwangslöschung unmöglich machen würde. Governance-Mode erlaubt eine Löschung durch dafür explizit autorisierte Rollen und bleibt gleichzeitig gegen reguläre, unautorisierte Löschversuche geschützt - welcher Modus für einen Objekttyp greift, sollte daher bewusst mitentschieden werden: Governance-Mode für alles, wofür ggf. eine Zwangslöschung möglich bleiben muss, Compliance-Mode nur für Objekttypen, bei denen das ausgeschlossen ist.
- **Löschgrund**: Ein Freitextfeld/eine Auswahl für den Grund der Löschung kann hinterlegt werden. Ob die Angabe eines Grundes **optional oder verpflichtend** ist, ist konfigurierbar je Installation/Objekttyp - für Installationen mit hohem Dokumentationsbedarf (z. B. öffentliche Verwaltung) empfiehlt sich eine verpflichtende Angabe, während andere Installationen das offener handhaben können. Der Löschgrund selbst wird - notwendigerweise getrennt vom gelöschten Objekt - im Audit-Trail (5.3) dauerhaft aufbewahrt, da genau das die Nachvollziehbarkeit *warum* etwas gelöscht wurde erst ermöglicht, auch nachdem der Inhalt selbst nicht mehr existiert.
- **Löschregister**: Jede physische Zwangslöschung wird zusätzlich in einem eigenen, separat gesicherten Löschregister vermerkt (Objekt-ID, Zeitpunkt, Grund) - unabhängig vom regulären Backup-Zyklus der übrigen Systemdaten. Zweck: Nach einer Wiederherstellung aus einem älteren Backup muss zuverlässig geprüft werden können, ob zwischenzeitlich zwangsgelöschte Objekte dabei unbeabsichtigt wiederhergestellt wurden, damit sie erneut

gelöscht werden können (siehe 10.4, Backup & Restore).

Einbindung in Prozesse

- Löschfristen und die Auslösung der Zwangslöschung lassen sich als Bestandteil einer **BPMN-Prozessdefinition** (7.1) modellieren - z. B. ein automatischer Prozessschritt (Automatic Task), der nach Ablauf der Frist die Löschung anstößt, optional mit vorgeschaltetem Genehmigungsschritt (Manual Task) oder Vier-Augen-Freigabe (4.3) vor der eigentlichen physischen Löschung, falls das für einen Objekttyp gewünscht ist.
- Alternativ ist die Zwangslöschung auch unabhängig von einem expliziten BPMN-Prozess direkt am Objekt/Objekttyp konfigurierbar - der Prozess-Weg ist eine Option für komplexere Fälle, keine Voraussetzung für die Grundfunktion.

Löscherinnerung (optional)

- Optional aktivierbar (Erwartung: eher seltener genutzt, aber unterstützt): Eine konfigurierbare **Vorlaufzeit** vor dem eigentlichen Löschzeitpunkt löst eine Benachrichtigung über den Notification Service aus, sodass berechtigte Personen die anstehende Löschung noch einmal prüfen und - falls nötig - stoppen oder die Frist anpassen können, bevor die Löschung tatsächlich stattfindet.
- Technisch dieselbe Timer-/Boundary-Event-Mechanik wie bei der SLA-Zeitüberwachung in Prozessschritten (7.1) - kein separater Mechanismus, sondern dieselbe BPMN-Timer-Infrastruktur, nur für einen anderen Zweck genutzt.
- Ohne Reaktion auf die Erinnerung läuft die konfigurierte Löschung nach Ablauf der Frist regulär weiter - die Erinnerung ist eine zusätzliche Kontrollmöglichkeit, kein Vetorecht, das die Löschung automatisch verhindert.

5.3 Audit-Log

- Wie in 3.4 beschrieben: jedes Event wird unveränderlich, mit Zeitstempel, handelndem Nutzer, betroffenem Objekt und Aktion gespeichert.
- Empfehlenswert: kryptografische Verkettung der Einträge (Hash-Chain), damit nachträgliche Manipulation auch bei DB-Zugriff erkennbar wäre.
- Auswertbar über eigene Admin-UI-Sicht (Audit-Trail je Dokument/Ordner/Nutzer), exportierbar für Prüfungen.

5.4 Reporting & Forensische Nachverfolgung

Der Reporting Service baut auf dem Audit-Event-Strom (3.4/5.3) auf und bedient zwei unterschiedliche Anwendungsfälle:

a) Standardberichte

- Vordefinierte, konfigurierbare Berichte (z. B. Dokumentenaufkommen pro Zeitraum/ Ordner, offene Workflow-Aufgaben, Lizenzauslastung, Speicherverbrauch je Backend, Nutzeraktivität).
- Als Read-Modell umgesetzt: der Reporting Service konsumiert den Event-Strom und hält eigene, für Auswertungen optimierte Aggregationen (siehe 3.1) - kein Live-Join über fremde Schemata.
- Exportierbar (CSV/PDF), planbar (regelmäßiger Versand über den Notification Service).

b) Objektbezogene Nachverfolgung (Forensik-Trace)

- Gezielte Abfrage: „Zeige alle Aktionen, bei denen Objekt X beteiligt war" - wobei X ein **Nutzer**, ein **Dokument**, ein **Ordner** oder z. B. eine **Rolle** sein kann.
- Zentraler Anwendungsfall: **kompromittierter Account**. Bei Verdacht auf Kompromittierung eines Nutzerkontos lässt sich lückenlos nachvollziehen, welche Aktionen dieser Nutzer **ab einem bestimmten Zeitpunkt** (z. B. dem vermuteten Kompromittierungszeitpunkt) durchgeführt hat - inklusive aller gelesenen, geänderten, heruntergeladenen oder gelöschten Dokumente, Berechtigungsänderungen, Force-Unlocks etc.
- Umsetzung: Da jedes Audit-Event sowohl den handelnden Nutzer als auch alle betroffenen Objekte referenziert (5.3), kann der Trace beidseitig gefahren werden - "alle Aktionen von Nutzer X" ebenso wie "alle Nutzer, die je auf Dokument Y zugegriffen haben".
- Zeitfenster-Filter (ab/bis) und Aktionstyp-Filter (nur Lesezugriffe, nur Änderungen, nur Downloads, ...) sind Kernbestandteil dieser Ansicht, damit ein Vorfall gezielt eingegrenzt werden kann.
- Ergebnis ist als Bericht exportierbar (z. B. für eine Sicherheitsuntersuchung oder Meldung an Datenschutzbeauftragte) und selbst wieder als Zugriff auditiert (wer hat wann welchen Trace abgefragt).
- Sinnvolle Ergänzung: automatisierte **Anomalie-Hinweise** (z. B. ungewöhnlich hohe Downloadzahl in kurzer Zeit durch einen Nutzer) als optionaler, regelbasierter Vorstufen-Mechanismus zur manuellen forensischen Analyse - kein Ersatz dafür, aber ein Trigger,

überhaupt genauer hinzuschauen. **Entscheidung für diese Konzeptversion**: Umsetzung zunächst ausschließlich über konfigurierbare statische Schwellwerte (z. B. "mehr als X Downloads durch denselben Nutzer in Y Minuten"); KI-/ML-gestützte Anomalieerkennung ist bewusst **out of scope** und wird nicht Teil des Kernsystems - eine spätere Ergänzung bleibt durch die "Dazustellen"-Architektur (1., 3.8) jederzeit möglich, ohne dass sie jetzt mitgeplant werden muss.

5.5 Feingranulares User-Tracking (Session-Fingerprinting)

Ergänzend zum regulären Audit-Log (5.3), das primär auf Aktionen bezogen ist, kann pro Nutzer ein **erweitertes Tracking** aktiviert werden, das über die reguläre Aktionsprotokollierung hinausgeht.

- **Standardmäßig nur für den Superuser aktiv** (4.6), für alle anderen Nutzer individuell zuschaltbar (z. B. bei konkretem Verdacht, für besonders privilegierte Konten, oder vertraglich/regulatorisch gefordert für bestimmte Rollen).
- Erfasst zusätzlich zum regulären Audit-Ereignis: vollständige Session-Metadaten (u. a. Client-IP, Geräte-/Browser-Fingerprint, verwendetes Endgerät soweit ermittelbar, Authentifizierungsmethode, Zeitpunkt/Dauer der Session, ggf. bekannte Netzwerk-/Standortinformationen), sodass sich im Bedarfsfall nachvollziehen lässt, von welchem Rechner/Kontext aus ein Konto tatsächlich genutzt wurde - zentral etwa für die Aufklärung, ob und von wem ein Konto kompromittiert oder durch Dritte mitgenutzt wurde.
- Läuft über denselben Audit-/Event-Mechanismus (3.4/5.3), aber mit eigener, feingranularerer Ereignisklasse, damit reguläre Audit-Auswertungen (5.4) nicht standardmäßig mit diesem deutlich größeren Datenvolumen belastet werden.
- Da hierbei besonders sensible personenbezogene Daten anfallen (Geräte-/Standortdaten), gilt ein eigenes, engeres Zugriffsregime auf diese Tracking-Daten (nur wenige, explizit berechtigte Rollen - Zusammenspiel mit 4.6) sowie eine **gesonderte, konfigurierbare Aufbewahrungsdauer** (**Standardwert: 7 Tage**), die kürzer bemessen ist als die reguläre Audit-Log-Aufbewahrung, sofern datenschutzrechtlich keine längere Aufbewahrung gerechtfertigt ist (Abwägung analog zu 5.2) - nach Ablauf werden die feingranularen Tracking-Daten automatisch gelöscht, unabhängig vom regulären Audit-Ereignis, das gemäß den allgemeinen Fristen (5.2) bestehen bleibt.
- Aktivierung/Deaktivierung des Trackings für einen Nutzer ist selbst ein auditierter, sensibler Vorgang und kann optional dem Vier-Augen-Prinzip unterliegen (4.3).

5.6 Records Disposal & Long-Term Archiving

In addition to the regular retention periods (5.2), the system supports regulated **records disposal**: the transfer of documents/structures whose active processing phase has concluded into a separate long-term archive (which, via the storage abstraction layer, 3.6, can itself reside on its own backend - potentially cheaper or redundant in a different way).

- The records disposal process is technically closely related to the migration process (7.2: lock → copy/transfer → verify → release in the archive → deletion in the active system after the transition period) and likewise runs as an audited, resumable workflow via the Workflow Engine.
- Triggering is configurable: automatically after the retention/active phase of an object type expires, or triggered manually.
- **Mandatory conversion into an immutable archive format**: Before transfer to the long-term archive, documents are automatically converted into a format that remains readable long-term and can no longer be altered unnoticed afterward - the original format (e.g. .docx) is not sufficient for this, since it would remain editable with the original application.
    - **PDF/A** as the standard target format for individual documents - uses the same conversion function that the Rendering/Preview Service (3.7) already provides for regular PDF/A conversion, applied here mandatorily rather than optionally.
    - **XDOMEA** as the target format for structured files/cases (circulation folders, 2.3) - the established German administrative standard for exchanging and handing over files/cases to archives (e.g. federal/state archives), including file structure, metadata, and referenced documents in a single, standardized exchange format. Particularly relevant in the public-sector context, where XDOMEA is already expected for handovers to archive administrations - ensures compatibility with existing public administration archiving processes without inventing a proprietary, incompatible records-disposal standard.
    - For circulation folders, this conversion consistently accesses the completion snapshot (2.3) - what gets archived is the state fixed at case completion, not a possibly further-modified original.
    - The original format (e.g. the original .docx version) is retained in addition to the archive-format conversion (no replacement), unless configured otherwise separately - the conversion supplements the archiving instead of losing information.
- **Optional encryption prior to records disposal**: Documents can be encrypted with a dedicated key, separate from ongoing operations, before transfer to the long-term archive,

so that a data leak in the archive (which is typically checked/maintained less often than the active system) does not immediately result in readable content.
    - **Key management is deliberately handled externally**, not within the system itself: the keys needed for encryption/decryption are not stored inside the DMS database or a DMS-owned schema, but in a separate, established key-management format - e.g. a **KDBX file** (KeePass database format) - which is backed up, stored, and protected with its own master password/access protection independently. This means a compromised DMS/archive alone is not sufficient to obtain the keys, since they reside in a completely separate storage location. Larger installations can also connect a full external KMS/HSM instead of the file-based variant - the KDBX variant is the simple, resource-efficient default path, not a requirement.
    - Configurable per object type/tenant, whether and with which scheme encryption is applied (e.g. an individual key per document vs. a shared archive key per time period - a trade-off between effort and degree of isolation in case of damage).
- Disposed documents remain findable/traceable via metadata and the audit trail (a reference in the active system indicating where the original is located), even though the content itself is no longer in the active storage target set - important for the ability to provide information despite the outsourcing. During the transition period (7.2), disposed elements additionally remain directly searchable/viewable in their own **records disposal special area** (2.5), not just indirectly via audit-trail references.
- Retrieval from the archive (if needed again after all) is a separate, likewise audited process with decryption only for authorized roles.

---

6. Diagnostics & System Tools

6.1 Central Query & Trace Console

A central, powerful query interface spanning the entire system - intended both for error analysis/support and so that administrators or integrators can formulate their own monitoring sensors (10.1) or evaluations themselves, without being limited to predefined reports (5.4a).

Basic Principle

- A dedicated **Query & Trace Service** that provides a unified query layer via the event bus

(3.4) and the read models of the other services (audit, reporting, monitoring) - no direct access to other services' DB schemas (the principle from 3.1 remains intact), but rather a federated view via the interfaces/read models provided for that purpose by the owning services.

- The query language is **deliberately powerful and unified for both read and write cases** - this is intentional, not an oversight: this power is a prerequisite for the console to actually serve as a full-fledged diagnostics/debugging tool, not just a limited reporting frontend. Syntactically, the language is specifically modeled **on PostgreSQL's SQL dialect (psql)** - a natural choice, since Postgres is already the underlying database (3.1); administrators familiar with `psql` will find their way around immediately instead of having to learn a completely proprietary notation. Technically, this deliberately does **not** involve writing a parser from scratch, but instead reuses and extends the real PostgreSQL parser (specifically via the Python library pglast/libpg_query, see 1a) - the DMS-specific additional constructs for trace/hierarchy/event queries (e.g. traversal of folder hierarchies, event time-window operators) are added as an extension of the resulting AST, instead of reproducing PostgreSQL grammar. This allows both arbitrary trace/evaluation queries ("show all events involving document X in the last 90 days, grouped by user") and targeted data manipulations ("reset attribute Y on all documents of type Z matching condition B") to be formulated. The language is thus also the foundation for formulating **custom, reusable sensor definitions** for monitoring (10.1), instead of having to write code for each one.
- **Functional scope strictly bound to the permissions of the executing person - no single query is possible without authentication** (no anonymous/technical access, not even read-only): The console executes every query in the security context of the logged-in user and applies exactly the same RBAC permissions, folder permissions, and scope locks (4.7) that otherwise apply to that person - a query can never see or change more than the executing person would be allowed to anyway. The activated superuser (4.6) is the only exception and can actually read and write without restriction via the console.
- On the manipulation side, in addition to the permission binding, several security levels apply mandatorily and cumulatively (not individually optional to disable in combination, but each level configurable on its own):
    i. **Protective switch ("manipulation mode")**: Write queries are disabled by default and must be explicitly unlocked for a limited time/session by an authorized role (comparable to a "break-glass" access, see also 4.6). Without an active protective switch, write expressions are already rejected at parse time, not only at execution.
    ii. **Dry run as default behavior**: Every write query is initially only simulated - the result

shows how many objects would be affected and a preview of the changes, before a second, explicit confirmation triggers the actual execution.

iii. **Optional four-eyes principle** (consistent with 4.3): Manipulating queries can - configurable per installation - require a second approval by another authorized person before they are actually executed after the dry run.

iv. **Mandatory four-eyes principle for tables/object areas marked as critical**: Independent of the general, installation-wide four-eyes configuration from point 3, individual particularly sensitive tables/entity classes (e.g. permission/role tables, license data, object-type/constraint definitions) can be marked as **mandatorily subject to the four-eyes principle** - this marking cannot be bypassed by a differing general configuration and **also applies to the activated superuser**, who would otherwise be able to act without restriction via the console. This means that even the most powerful access path remains additionally secured for the truly most critical data areas.

v. **Complete logging**: Every query (read as well as write), the executed query text, the result/impact, and the people involved (the executor, and any four-eyes approvers) are audited (5.3) - the console itself is thus subject to the same traceability that it provides for the rest of the system.

- Access to the console is itself RBAC-controlled and can be restricted granularly (e.g. "read-only", "domain X only", "no manipulation") - fits the concept of the scaled-down, domain-separated admin roles (4.6).
- The console can be used both via a UI component (part of the admin web interface, 8) and via the CLI tool (6.2) - the same query language and the same security levels apply equally in both access paths.

6.2 CLI Tool

A complete command-line tool for configuration, auditing, tracing, and debugging - conceptually modeled on tools like `oc` (OpenShift): a client that talks to the system's core APIs (via the API gateway/BFF, 3.5) and thus respects the same permissions and security levels as the web UIs.

- **Functional scope**: configuration import/export (7.3), queries via the Query & Trace Console (6.1), triggering/observing migration/transfer operations (7.2), management of object types/constraints/workflows, insight into registry/plugin orchestration status (3.2/3.8), license status (9.3), user/role management, triggering/monitoring backup and restore

operations including progress of the deletion reconciliation (10.4), delta/comparison runs between installations (7.5) with scriptable output.
- **Domain separation**: The CLI is built modularly, so that it can (as suggested) be organized into functional subcommands that can be rolled out/authorized separately - e.g. a leaner CLI subset only for auditing/reporting to business units, while administrative interventions (object types, migration, manipulation via 6.1) remain reserved for a narrower group of users.
- Authentication via the same mechanisms as the web UIs (4.4), including support for service accounts (e.g. for CI/CD pipelines that roll out configuration automatically).
- Scriptable/automatable (structured output formats such as JSON), so that CLI calls can be embedded in automation and deployment pipelines (e.g. automated rollout of a new installation from a configuration export, 7.3).

---

7. Processes: Workflow Engine, Migration, Configuration

7.1 BPMN-Based Workflow Engine

The Workflow Engine is based on **BPMN 2.0** (Business Process Model and Notation) as the modeling and execution standard, rather than a proprietary state-machine format - this allows both modeling custom processes directly within the system and the **import of existing BPMN models** created, for example, with external modeling tools. Concrete implementation based on **SpiffWorkflow** (see 1a) as a Python-native BPMN execution library.

Modeling & Import

- A **Process Designer** (a standalone frontend application, **not** part of the admin web interface, see 8) allows graphical modeling of business processes/workflows directly within the system - the result is a standard BPMN 2.0 XML document.
- **Import of existing BPMN files** (BPMN 2.0 XML) from external tools is supported; on import, validation is performed against the task types/connectors available in the system (e.g. whether referenced object types, folder targets, or instance targets for federated steps actually exist), with a clear error message for unresolvable references.
- Process definitions (imported or self-modeled) are part of the JSON/BPMN configuration

and can be exported/imported via the configuration export (7.3) - consistent with the existing cloning/transfer principle between installations.

Guiding Business Objects Through Processes

- A BPMN process automatically guides **business objects** - documents or circulation folders (2.3) - through the departments/roles/processing steps defined in the process, including the associated status transitions, permission checks (RBAC, 4.1), and constraint checks (2.2) at each step.
- **Manual Tasks (User Tasks)**: require an active action by an authorized person (approval, review, editing) - appear as a task in the user/reviewer UI (8).
- **Automatic Tasks (Service Tasks)**: are executed by the system itself without manual involvement (e.g. automatic forwarding after a deadline expires, automated constraint check, triggering a connector call) - standard BPMN concept, directly supported.
- **Signature Tasks**: a third task type that requires an electronic signature (3.10) from an authorized person on the respective business object before the process continues - technically a special form of the Manual Task, but modelable as a distinct type in its own right, since the required signature level (SES/AES/QES) is defined per process step.

Decision Logic via DMN Decision Tables (Update P12b-S1/user decision)

- In addition to pure BPMN gateways, the Workflow Engine supports **DMN 1.3** (Decision Model and Notation) for business decision logic that can be modeled more clearly as a table rather than as a nested gateway cascade (e.g. "which approval level depending on amount/object type/department").
- A **Business Rule Task** in the BPMN model calls a named DMN decision table, evaluates the current process/business-object attributes against its rules, and writes the result back into the process data - the subsequent process flow (e.g. a gateway) can branch based on it.
- DMN tables are edited **in the same Process Designer** as BPMN models (no separate tool), are part of the same configuration, and can therefore be exported/imported via the configuration export (7.3) just like process definitions themselves.
- Concrete implementation based on the DMN support of **SpiffWorkflow** (1a) - the same library that already handles BPMN execution, no additional third-party component needed.

Time Monitoring per Process Step (SLA Tracking)

- A **maximum permissible processing duration** can be configured for each individual process step (part of the process definition, not global for the entire process).
- If a step exceeds this duration, the system triggers a configurable escalation (e.g. notification to a supervisor role via the Notification Service, marking as overdue in the UI, optionally an automatic escalation branch within the BPMN model itself via a boundary timer event) - the goal is that no case goes unnoticed.
- **Regional business calendars for deadline calculation (Update P12b-S1/user decision)**: deadline calculation can optionally take a configured **business calendar** (non-working days) into account, instead of calculating exclusively with plain time/duration - a step with a "3 business days deadline" then does not run through a configured weekend/holiday. Deliberately implemented **as a freely configurable data structure** (a named list of non-working dates/rules, freely maintainable per installation, multiple calendars possible in parallel and assignable to a process/an installation) instead of a hard-coded region selection (e.g. German federal states) - saves development effort for every individual jurisdiction and does not tie the system to one region. Optionally, pre-built calendar templates (e.g. common holiday calendars) can be shipped as a starting point, but remain ordinary, freely modifiable configuration data, not special logic. Without a configured calendar, the previous behavior (plain time/duration) remains the unchanged default.
- The actual processing duration per step is recorded as an audit event (5.3) and is thus also the basis for standard reports (5.4a) and the forensics/trace console (6.1) - e.g. "average processing duration per step/department" as a reporting metric.

Multi-Instance Processes (Cross-Installation Process Steps)

- A process step can be defined as a **handover to one or more other installations** - handled technically via the Federation Hub (7.4); the business object (document/circulation folder) is handed over to the target installation(s) configured in the process step.
- **The target(s) are fixed in the process step at process-modeling time** (no runtime selection by the processing person) - the installations available for selection come from the **Federation Hub's address book** (7.4).
- **This option is only available if the installation actually participates in a Federation Hub** (7.4 is optional) - if no hub is configured/no address book exists, the Process Designer does not even offer federated process steps as a selectable option, rather than displaying an option that would lead nowhere.
- After handover, process progress on both sides (the sending as well as the receiving installation) continues to be tracked locally; the return of the processing result again runs

back to the original process via the same Federation Hub mechanism, provided the process specifies this (e.g. for a review by another authority followed by a return).

7.2 Migration/Transfer (Export and Import of Structures)

Adoption of the proposed process, supplemented with safeguards:

1. **Lock** the source folder (including all subfolders) - no write access during migration.
2. **Copy** the content including metadata, version history, permissions, and audit references to the target system.
3. **Verification**: checksum/hash comparison per document (not just a count comparison), reconciliation of metadata and version completeness.
4. **Release in the target system** after successful verification.
5. **Deletion in the source system** only after a configurable transition/retention period has elapsed (a safety net for rollback).

In addition:

- The entire process itself runs as an auditable, resumable workflow via the Workflow Engine (not as a special case alongside it) - in case of abort, resuming from the last confirmed step is possible instead of having to start over.
- **Dry-run mode**: simulation of the transfer (including constraint checking in the target system, e.g. are matching object types present?) without actual data movement.
- Migration/transfer is a separate, optionally licensable component and can itself be subject to the four-eyes principle (see 4.3).

7.3 Configuration Import/Export (JSON)

- Complete system configuration (object types, constraints, workflows, role/permission templates, four-eyes settings per action type, UI customizations) exportable as a JSON document.
- Re-import into another (or the same, e.g. staging→production) system is possible - thus also the basis for system cloning/test environments.
- Versioning of the configuration schema itself, so that an export from an older version can be imported into a newer one (with migration logic for schema changes).

7.4 Cross-Tenant Workflows (Federation Hub)

In addition to pure migration (7.2, which describes a one-time, final transfer), the system supports **ongoing business processes that function across the boundaries of fully independent installations** - e.g. an approval step that resides with a partner organization with its own installation, or a document that travels between several organizations as part of a process, without any of the participating installations having control over the other.

Federation Hub as a Separate, Standalone Building Block

- A dedicated **Federation Hub Service** (deliberately not part of a single installation, but an independently operated mediation service, possibly provided by a neutral operator) takes on two tasks, analogous to the registry principle (3.2) but across installations:
  - **Address book**: installations register with the hub using a public identifier and the process/document types for which they are available as participants in federated workflows - no installation needs to know the internal addresses/endpoints of another directly.
  - **Switching center**: the hub mediates process steps/document handovers between more than two participating installations without permanently storing document content itself - it forwards handovers and only logs metadata of the mediation process (who handed over what to whom, and when), not the document content itself.
- Each installation remains fully autonomous with regard to its own data (the principle from 3a remains intact) - a federated workflow step is, from the perspective of each participating installation, a regular, locally audited inbound/outbound item (comparable to a mail room inbound item from outside), not third-party access to its own database.
- Federated workflow steps are part of the normal Workflow Engine definition (7.1): a process can contain a step that means "handover to an external installation via the Federation Hub", including a wait state until the response.

Operating Model

- Since the software is open source, **both operating variants of the Federation Hub are explicitly provided for and equally supported**: a customer/operator can run their own Federation Hub themselves (e.g. for federation between their own multiple installations, 3a), or an operator can also provide a hub for other, external installations (e.g. a consortium of several organizations that agree on a jointly operated hub).
- The developer side/the project itself does **not** operate a **public production Federation**

Hub as its own offering - a hub centrally provided by the project exists, if at all, for internal purposes or testing, not as part of the product for end customers. This is a deliberate consistency decision in keeping with the open-source character: federation remains entirely in the hands of the operators, with no central dependency on project infrastructure.

Version Compatibility Between Installations

- Since installations with different version states can participate in a federation, each installation reports to the hub upon registration not only its own version, but also a **declared compatibility range** - e.g. "I am version 9.3, but can also handle federated workflows with versions from 8.2 onward".
- When mediating a federated process step, the hub checks whether the compatibility ranges of the participating installations overlap, and rejects the mediation with a clear error message if not - rather than mediating a step that one side cannot process correctly at all.
- This compatibility range is part of the already-versioned configuration schema (7.3) and is explicitly maintained with every release of the system (which older versions remain federation-compatible), not assumed implicitly.

Security & Governance

- Participation in the hub is opt-in per installation and per process/document type - an installation explicitly determines with which other (known, trusted) installations it conducts federated processes, not an automatic open network.
- Transferred documents/data are encrypted end-to-end between the participating installations (the hub itself cannot view the content, only mediate it) - technically related to the optional encryption during records disposal (5.6).
- Every handover is fully audited (5.3) on both participating sides; additionally - as with regular migration - a verification (checksums) can be performed before the handover step is finally completed.

7.5 Delta/Comparison Function Between Installations

In addition to plain configuration export/import (7.3), the system supports a targeted **comparison of two installations** against each other - intended, for example, for drift detection between staging and production, for reconciling several installations managed by the

same operator (3a), or to see what would actually differ before a planned configuration takeover.

Basic Principle: Baseline Instance vs. Comparison Instance

- One installation is designated as the **baseline instance** (reference), one or more others are **compared against it**. The result is a directed delta report ("what deviates in the comparison instance from the baseline instance"), not merely an undirected similarity check.
- The comparison works on the basis of the existing **configuration exports** (7.3) of both installations, not via direct live access by one installation to another's database - this remains consistent with the principle of complete installation isolation (3a). The two exports can be provided manually or - if both installations participate in a shared Federation Hub (7.4) - retrieved automatically via the controlled channel there; however, a hub is not a prerequisite for the function.
- The function is **purely read-only/diagnostic**: it changes nothing on either of the two installations. If identified differences are actually to be adopted, this happens deliberately via the regular, already secured configuration import (7.3) - with all the protective mechanisms that apply there (e.g. four-eyes, if configured) - not via a separate, unchecked synchronization path.

Configurable Comparison Scope (Default: Everything)

- Analogous to the already established pattern for monitoring (10.1: "everything or nothing, granularly adjustable"), the following applies here: by default, the comparison covers **all** comparable categories - object definitions (2.2), BPMN processes (7.1), role/permission templates (4.1), four-eyes configuration per action type (4.3), AD group mapping rules (4.4), constraint definitions, UI customizations, monitoring sensor configuration (10.1) - and can be narrowed down to individual categories if only a subset is of interest (e.g. "only compare object definitions and processes, ignore permissions").
- Purely installation-specific data (e.g. license status, 9.1, or the concrete registry reachability, 3.2) is fundamentally excluded from the comparison, since a difference here does not represent a configuration deviation, but expected, installation-specific behavior.

Configurable Ignore Regex for Object Names

- Since objects (object types, processes, roles, etc.) can have differing naming conventions

between installations without this representing a real substantive difference - exactly the scenario mentioned in the example, where `100_testobjekt_typ_alpha` on one installation and `101__testobjekt_typ_alpha` on the other actually refer to the same object -, an **ignore regex** can be defined that is applied to both names before the actual name matching (normalization/cleanup, e.g. removing numeric prefixes and inconsistent separators), so that the cleaned-up names are recognized as identical even though the raw values differ.
- This ignore rule concerns **exclusively the mapping of which object counts as "the same" on both sides** (object identification for the matching) - the actual substantive comparison of the remaining attributes of the object thus matched (constraints, permissions, field structure, etc.) continues to take place fully and independently of this, so that substantive deviations do not go unnoticed despite the "same" name.
- Configurable both **globally** (one ignore regex for all categories) and **per category** (e.g. different naming conventions for object types vs. processes), since naming schemes typically differ between object kinds.
- Without a configured ignore regex, exact name comparison applies as the default behavior - the function is therefore already usable without any configuration, the ignore regex is an optional refinement.

Result Presentation

- Delta report with clear categorization: **present only in the baseline instance** (missing in the comparison instance), **present only in the comparison instance** (additional, not in the baseline), **present in both, but substantively deviating** (with detail view of which attributes specifically deviate), **identical** (the predominant normal case, usually displayed collapsed/summarized rather than listing every identical object individually).
- Available both via the admin UI (8) as a visual presentation and via the CLI tool (6.2) in structured, scriptable output - relevant, for example, for automated drift checks in a CI/CD pipeline before a planned configuration rollout to multiple installations.

---

8. UI Layer

- **User Web Interface**: document/folder navigation (including creating/renaming/moving folders, not just browsing), metadata display/editing per document (attributes according

to object type, 2.2/4.5 - the object type definition itself remains the responsibility of the admin UI, its values on the specific document belong in the user UI), search, upload, workflow interaction (approvals, tasks), preview.

- **Layout guideline (default arrangement)**: a three-part main area, modeled on familiar desktop file-manager metaphors (recognizability lowers the entry barrier for end users who are already familiar with this kind of interaction from their operating system) - the explorer on the left, the document tabs in the upper right (several open documents/folders in parallel, analogous to browser/editor tabs) above the document preview (2.4/3.7), and below that the metadata panel of the document selected in the active tab. The tab selection controls the metadata panel and preview in sync - when the tab changes, both change together. This arrangement is the **factory default**, not a fixed requirement (see flexible workspace below).
- **Flexible, customizable workspace (editor metaphor, analogous to VS Code)**: beyond the default arrangement, the main workspace can be freely rearranged - panels (explorer, metadata, preview, further document tabs) can be arranged side by side or stacked, split off into their own areas, or merged back together, comparable to the dockable editor/panel layout of Visual Studio Code. In particular, several documents can be made visible simultaneously side by side or stacked, instead of only being shown one after another via tabs. This customization is a pure display preference (analogous to the explorer tree view, 2.2a/8) and is remembered **locally per device**, not across devices like the color scheme (below) - a workspace layout is typically tied to the screen size/usage context of the respective device. A "reset to default arrangement" is possible at any time.
- **Deliberately deferred: free CSS/theme customization by end users** - beyond the four color schemes (below), users are not offered their own CSS customization. Technically this would be possible without much additional effort (the same foundation as the workspace flexibility above), but carries the risk of overwhelming inexperienced users with too many degrees of freedom or leading to inconsistent-looking, hard-to-support interfaces - therefore remains a later, non-binding extension candidate rather than part of this concept version.
- Additionally, at the very left edge, a narrow, icon-based navigation bar for cross-cutting functions (e.g. tasks/inbox, search, profile/settings) - deliberately outside the three-part main workspace, since it does not belong to the actual document workspace.
- **Selectable explorer presentation**: in addition to the classic list view, the user can

switch the explorer to a **tree view** that structurally reflects the actual folder hierarchy - a pure display preference (analogous to the color scheme below), not a system default; which presentation is used is decided by each person individually. In both views, the **class icon** (2.2a/2.2b) of the respective object-type assignment is displayed before the name of every folder/document, so that different classes can be distinguished at a glance without having to display the object-type name.

- **Admin Web Interface**: user/role management, object-type/constraint editor, license overview, audit-trail view, configuration import/export, registry/service overview (which services are active, licensed, health status). The workflow/process designer is **not** part of the admin UI (see separate point below).
  - **Layout guideline**: a classic management dashboard - a left, optionally expandable/groupable navigation sidebar (areas such as users & roles, object types, registry, ...; further subdivision as more areas are added), main area on the right shows the respectively selected function.
  - **Object-type/layout editor as a guided GUI, not as text/JSON input**: the object-type editor (2.2/2.2a/2.2b) is a form-based wizard - the administering person selects attributes, assigns a display name per attribute, and marks required fields, instead of editing the underlying JSON structure (2.2) by hand. From these inputs, a default layout ("smart layout") is automatically generated. A supplementary **Layout Designer** (its own admin UI area) allows targeted fine-tuning of this generated layout - column/row arrangement of the fields - both for the metadata display and for the independently adjustable search and upload masks of the same object type (2.2b).
  - **Managing multiple installations from one admin UI**: since an installation (3a) has its own, fully isolated identity, the admin UI maintains a list of configured installations (each with its own gateway endpoint) and its own session per installation - switching between installations already logged in once does not require logging in again, as long as the respective session remains valid. No single sign-on across installation boundaries (would contradict the deliberate isolation from 3a), but also no need to run the admin UI multiple times or log in again for every installation each time.
- The **Process Designer** (BPMN modeling, 7.1) is a **standalone frontend application**, not part of the admin web interface - its own functional focus (process modeling rather than administration), its own technical needs (a BPMN modeling component, e.g. `bpmn-js`, see 1a), deployable independently of the other UIs.
- Conceivable going forward: a dedicated **Reviewer/Approval UI** (a lean focus purely on approval tasks, including four-eyes cases) as well as a **Migration Console** for transfer

operations.

- **Color scheme**: every web UI supports **light**, **dark**, **high contrast** (accessibility), and **automatic** (follows the device's system setting). The preference is **stored in the user profile**, not just locally in the browser, and takes effect **across devices** - if the same person logs in on another device, the same setting applies there as well.
- Customizability: configurable views/forms per object type (which attributes are displayed/editable where), branding/theming, role-dependent dashboards where applicable.
- **Bulk editing of metadata (Update P12b-S1/user decision)**: several selected documents/folders can be assigned the same attribute values together in one step, instead of having to open each object individually - runs via the same rule engine/constraint check as a single change (4.5), and thus fails in a targeted way for individual, non-matching objects within the selection instead of silently bringing the entire selection into inconsistent states; the result (success/failure per object) is clearly summarized and displayed upon completion.
- Technically implemented as separate frontend applications against the respective BFFs (see 3.5), so that UI extensions are likewise independently deployable.

Rendering Strategy: Client-Side Rendering (Decided)

- For the authenticated core application (user/admin/reviewer/process-designer UI), **client-side rendering (CSR)** is used, i.e. React/Next.js primarily as an SPA framework rather than with full server-side rendering throughout - **decision: deliberately made in favor of lower operational/resource load**, no longer merely a recommendation:
  - **SEO is irrelevant**: the entire application sits behind authentication (4.4) - the classic SSR advantage (better search-engine indexing, faster first contentful paint for anonymous visitors) does not apply to the vast majority of the application.
  - **Avoids an additional Node runtime layer in the backend**: since the backend services run in Python (1a), full SSR throughout would mean that a separate Next.js Node process would have to server-side reload data from the Python BFFs/APIs on every page request and render HTML from it - an additional operational component (its own scaling, its own monitoring, its own source of errors) with no clear benefit when the content is personalized/permission-dependent and not cacheable anyway. Foregoing this layer saves exactly this overhead.
  - The CSR application talks directly to the Python BFFs/API gateway (3.5) as a JSON API; Next.js is used essentially as React build tooling and a routing framework (possibly with static export), not for dynamic server-side rendering per request.

- **Targeted exceptions where SSR/SSG remains sensible**: publicly reachable, non-personalized pages outside the actual application (login page, possibly a public status page, documentation/help pages) - there, static or server-side rendering is unproblematic, since no per-request Python data access is needed.

---

9. License System

9.1 License Dimensions

A license can encompass several, independently configurable dimensions:

Dimension	Characteristics
Number of users	either **concurrent users** (concurrent sessions) or **maximum number of users** (named accounts) - fixed per license which model applies
Application components	individual services/modules are separately licensable (e.g. CMIS connector, migration service, workflow automation)
Storage volume	upper limit in GB/TB, optionally unlimited
Document count	upper limit on documents, optionally unlimited

Each dimension can be individually set to "unlimited" - a license with unlimited users and documents must be representable just as much as a tightly limited one.

9.2 License Service

- A standalone service that manages and validates license keys/certificates (e.g. a signed license file containing all dimensions and the validity period).
- Continuously checks (not only at startup) current usage against limits, e.g. via events from the audit/usage stream (current concurrent sessions from the Auth Service, document count from the Document Service, aggregated storage consumption).
- Publishes status changes (limit reached, license expires in X days, license invalid) as events

- consumed among others by the Registry Service (see 3.2b) and by the Notification Service (admin notification).

9.3 Behavior With Missing/Invalid License

- The Registry Service supplies each registering service with the license status of its component.
- Without a valid license for a component: **demo mode** (functionally restricted, e.g. read-only access, visible watermarks/notices, time-limited) or complete lockout - configurable per component which behavior applies.
- Exceeding a usage limit (e.g. document count) does not retroactively block existing data, but prevents new creation until the limit is met again or the license is expanded.
- The license status is also viewable at any time in the admin UI (see 7), including remaining term and utilization per dimension.

---

10. Operations & Extensibility

10.1 Monitoring & Observability

Monitoring is treated as a standalone, configurable building block - not as a fixed "on or off", but controllable granularly down to the level of individual measurement points.

Sensor Concept

- Every service brings along a set of **sensors**: individual, clearly named measurement points (e.g. `document.upload.duration`, `permission.check.latency`, `storage.backend.s3-primary.write.errors`, `registry.service.heartbeat.miss`, `license.usage.documents.current`).
- Sensors are grouped functionally (e.g. by service, by category such as performance/errors/capacity/security), so that they can be enabled/disabled clearly in the admin UI without having to go through each individual sensor manually.
- Every sensor knows its own cost (CPU/IO overhead, label cardinality) - high-resolution or expensive sensors are clearly marked, so that an "everything on" setting does not accidentally place a noticeable burden on the system.

- **Principle of "maximum coverage, but only when needed"**: fundamentally, everything that can be meaningfully measured should be measurable - there is no artificial limitation of the sensor catalog. Resource conservation does not come from fewer available sensors, but from the fact that a sensor **only generates data at all** when it is actively switched on in the Monitoring Service: if a sensor is disabled, the recording in the respective service is already omitted (no generation, no intermediate storage, no feeding back to the metrics endpoint) - not just the display/export of the data. Anyone who deliberately keeps monitoring lean thus actually saves system resources, rather than merely reducing visibility.

Configurability (Default-On/Default-Off + Fine-Grained Control)

- Global base setting per installation: **"monitor everything"** (all available sensors active) or **"monitor nothing"** (monitoring completely disabled) as a starting point.
- Building on this, targeted overrides: individual sensors or entire sensor groups can be switched on/off independently of the base setting (e.g. base setting "nothing", but specifically activating only the Storage Service's sensors for a current capacity problem; or base setting "everything", but deliberately switching off expensive tracing sensors during continuous operation).
- Configuration is - like other system settings - part of the JSON configuration export (7.3) and individually adjustable per installation; changes take effect without restarting the affected services (sensors are switched on/off at runtime).
- The sensor configuration itself is audited (who activated/deactivated which sensors when) - relevant, since disabling security-relevant sensors is itself a security-relevant act.

Architecture & Integration With Standard Solutions

- Every service exposes its active sensors via a unified **metrics endpoint** in Prometheus exposition format (de facto standard, directly scrapable).
- In addition to service discovery (3.2), the Registry Service maintains a **sensor registry**: which services offer which sensors, which are currently active - this allows Prometheus scrape configuration or CheckMK discovery to be largely derived automatically from the registry, instead of maintaining it manually for every new service (fits the "add-on" principle: a new service brings its sensors along and is immediately monitorable).
- **Prometheus**: primary scrape consumer of the metrics endpoints.
- **Grafana**: consumer of Prometheus (or optionally directly supported data sources) for dashboards - the system itself does not deliver its own Grafana-replacement UI, but provides sensible default dashboard definitions as exportable JSON templates (a starting

point, not mandatory use).
- **CheckMK**: integration via its standard Prometheus special agent or alternatively a dedicated CheckMK-compatible check plugin, so that classic IT monitoring landscapes (alerting, escalation, SLA reporting) can integrate the system without having to maintain a parallel world to Prometheus/Grafana.
- The system does not thereby commit itself to a particular monitoring tool, but delivers standard-compliant metrics that are consumed by common solutions - custom integrations of further tools are fundamentally possible via the same metrics endpoint, without changes to the DMS itself.

Distinction From Audit/Tracing

- Monitoring (sensors/metrics) is deliberately separated from the audit trail (5.3): metrics are aggregated, anonymous/technical, and primarily intended for operations/capacity planning/alerting, while the audit trail is personally identifiable, complete, and immutable for traceability/forensics (5.4). Both streams can use the same event bus (3.4) as a technical foundation, but serve different purposes and are subject to different access/retention rules.
- Distributed tracing (e.g. OpenTelemetry) across service boundaries supplements the sensor metrics with the path of individual requests (gateway → Document Service → Permission Service → Constraint Engine → Audit Service) - likewise configurable in a sensor-like manner (e.g. active only for a portion of requests to limit overhead: sampling rate as its own configuration parameter).
- Central log aggregation (e.g. via event bus or log shipping) supplements metrics and traces with full-text error details.

10.2 Deployment & Scaling

- Every service is independently containerized and horizontally scalable (multiple instances per service type, the registry distributes load/reachability).
- New service types (further connectors, new rule types, new UI modules) are "added on" by deploying a new instance and are automatically integrated via the registry - without downtime or intervention in existing services.
- Placement and ongoing redistribution of these instances is handled automatically and time-profile-aware by the Plugin Orchestration Service (3.8), rather than requiring operators to manually determine which service runs on which server.

- **Distinction from classic EIM platforms**: systems like nscale are historically not designed throughout for stateless horizontal scaling of every individual component - individual core components cannot be freely multiplied there. The concept described here structurally avoids this by designing every service to be stateless from the ground up (state resides in the shared DB/event bus/storage, not in the service process), and any number of instances can run in parallel.

10.3 Security Aspects (Supplementary)

- Virus scanning is mandatory before releasing an upload - uploads that fail land in their own quarantine area (2.5) instead of being discarded or silently rejected.
- Encryption of data at rest and transport encryption (TLS) between all services.
- Separate DB credentials per service (see 3.1) also as a security measure, not merely an architectural principle.

10.4 Backup & Restore

Backup/restore is planned as a standalone part of the concept considered from the outset - not a subsequently bolted-on operational topic, precisely because the distributed architecture (multiple storage locations, shared DB with separate schemas, event bus) would make uncoordinated backup of individual components particularly error-prone.

What Is Backed Up

- **Shared Database (3.1)**: complete, consistent backup of all schemas together (not separated schema by schema at different points in time, otherwise the owner-schema relationships would lose their consistency) - classically via `pg_basebackup` /continuous WAL archiving for point-in-time recovery, not just periodic full dumps.
- **Storage backends (3.6)**: backup in addition to the already existing redundancy (multiple backends written to simultaneously) - redundancy protects against backend failure, but not against logical errors (accidental/malicious deletion, a bug in a service), which would otherwise affect all redundant copies equally. Therefore, an additional, genuine, time-shifted backup (e.g. object versioning with retention of deleted/overwritten versions, or a separate backup target outside the regular storage target set).
- **Event bus (3.4)**: depending on the chosen technology (NATS JetStream/Kafka), its own snapshot/retention mechanisms; since audit-relevant events already end up permanently in the Audit Service schema anyway (5.3), the event bus itself is primarily relevant for

short-term recovery/replay, not as the sole source of truth.

- **Configuration** (7.3 export) as an additional, independent recovery point - in an extreme case also allows a "rebuild" of the system configuration independent of a database restore.
- Keycloak (4.4) uses the same Postgres instance (3.1) and is thus automatically included in the regular DB backup - no separate mechanism needed.

Order of Restoration

The order is deliberately fixed to avoid inconsistencies between components (e.g. database entries referencing objects that do not yet exist in storage):

1. **Determine the consistency anchor**: the storage backup and database backup are fixed to a common, coordinated point in time (point-in-time consistency) - a restore to temporally different states of storage and DB is the main cause of inconsistencies and is excluded as an approach.
2. **Restore storage backends first** - the referenced objects must exist before the database (which references them) is put back into active use, otherwise a time window with "dangling" references arises.
3. **Restore the database to the same consistency point** (point-in-time recovery via WAL archiving).
4. **Perform the deletion reconciliation** (see below) - a mandatory step before the system is released again, not an optional follow-up.
5. **Event bus**, if applicable, reset to the same point in time or rebuilt (uncritical for most consumers, since they hold states that are reconstructable from the restored DB anyway - e.g. the search index, see point 7).
6. **Restart the registry, License Service, and remaining services regularly** - they re-register just as in normal operation (3.2), no special handling needed, since the registry state is continuously re-determined as "currently reachable" anyway rather than needing to be restored from a backup.
7. **Rebuild derived/reconstructable states last**: the search index (Search Service) does not need to be restored from its own backup, but can - if reasonable in terms of time - be re-indexed from the restored data set; this reduces the number of components that need their own, consistent backup at all.

Mandatory Deletion Reconciliation After Restore (Interaction With 5.2a)

The most important special case: a restore to a point in time *before* a mandatory deletion that has since taken place (5.2a) would unintentionally restore the document that legally should

have been deleted ("resurrection" of deleted data) - the system must actively prevent this, not leave it to operations staff.

- **Deletion register as a standalone record untouched by the restore**: in addition to the reduced regular audit log, every physical mandatory deletion (5.2a) is recorded in its own, separately and more frequently backed-up **deletion register** (object ID, deletion time, deletion reason) - this register is deliberately **not** reset to the same point in time as the rest of the database during a restore, but remains at its most current state (e.g. through separate, continuous backup/replication outside the actual restore process).
- **Automatic reconciliation as a mandatory step**: after every restore, the system automatically checks whether any of the restored objects are ones that, according to the deletion register, had already been mandatorily deleted **after the backup point in time but before the restore point in time**. For every case found, the physical deletion is **automatically executed again** - the same mechanism as for the original mandatory deletion (5.2a), including an audit entry noting that this occurred as part of a restore reconciliation.
- Only after this reconciliation has completed successfully is the restore considered complete and the system released again for regular operation - not before, in order to prevent someone from accessing, in the meantime, a document that should actually already have been deleted.
- The same logic applies analogously to regular (non-physical) deletions and records disposal (5.6) as well, even though the consequence of an accidental "restoration" is less severe there than for a legally mandatory deletion.

Execution

- Backup/restore operations themselves are audited (5.3) and automatically trigger a maintenance mode on restore, comparable to the not-shutdown maintenance state (4.8) - regular user access remains locked until the deletion reconciliation is complete.
- Restoration must be plannable to be testable/practiced (regular restore tests on an isolated test environment), to ensure that the order and the deletion reconciliation actually work in a real emergency - a backup that has never been successfully restored is not a reliable backup.

10.5 Uninterrupted Updates (Rolling Updates)

Updates should, wherever possible, not interrupt use of the application - this is not an

additional feature, but a direct consequence of the consistently stateless service architecture (guiding principle 1/6): since every service holds its state exclusively in the shared DB, event bus, and storage (never in its own process memory), an individual service instance can fundamentally be replaced without any business state having to be "carried along".

Basic Mechanism: Parallel Operation Instead of In-Place Replacement

- When updating a component, a **new instance in the new version is first started in addition to the running old instance**, rather than directly replacing the old one.
- The new instance registers with the Registry Service (3.2) and undergoes a health/readiness check there before it is considered ready for use at all.
- As soon as the new instance is recognized as ready, the registry begins routing **new** requests preferentially/exclusively to the new instance, while the old instance is put into a **draining status** - the same state that the Plugin Orchestration Service (3.8) already uses for gentle redistribution during load spikes. Technically this is the same drain mechanism, only used here for a different occasion (update instead of rebalancing) - not a second, separately maintained concept.
- In the draining status, the old instance no longer accepts new tasks, but regularly completes operations already in progress (e.g. a running BPMN process execution, a running storage write operation, an open signature task, 3.10) - the old instance is only terminated after all pending operations have fully completed.
- In case the new version turns out to be faulty after the switchover, the old instance remains addressable until the drain is complete - a fast switch-back (rollback) is thus possible as long as the drain of the old instance has not yet fully completed.

Limits: The Persistence Layer as a Special Case

- This procedure works completely without interruption for updates that involve **no change to the database schema or storage format** (pure code/logic changes to one or more services) - this is the normal case for smaller patches/updates and is expected as the standard procedure.
- For updates that require a **schema change to the shared DB (3.1)**, a more differentiated view applies:
    - Where possible, the established **expand/contract pattern** (also called "parallel change") is followed: a first migration step adds new structures (columns/tables) without removing existing ones ("expand") - old and new service versions can use the database in parallel during this time, since the old structures continue to exist. Only

after **all** instances have been updated to the new version and the old ones have been fully drained does a second, subsequent migration step remove the no-longer-needed old structures ("contract"). This means DB schema changes, too, can as a rule be carried out without interruption, but require two separate rollout steps instead of a single one.
    - The same principle applies analogously to changes to the **storage format** (3.6): new and old object formats coexist in parallel during the transition phase until all consumers have migrated to the new version.
    - It is deliberately not claimed that **every conceivable** structural change is possible without interruption - for a fundamental restructuring that cannot be meaningfully broken down into expand/contract, a short, planned maintenance window may be unavoidable in an individual case. That is then, however, a deliberately planned, clearly communicated exception (with advance notice, analogous to the maintenance mode during restore, 10.4), not the normal case.
- API compatibility between services during the transition phase is a prerequisite: since old and new service versions communicate simultaneously with other, not-yet-updated services for the duration of the drain, every service version within an update rollout must remain backward/forward compatible with the other (no breaking API changes within a single rollout operation) - the same basic discipline already described for version compatibility between federated installations (7.4), applied here at the service level within a single installation.

---

11. Architecture Overview (Diagram)

Rough depiction of the most important services and their communication paths. Deliberately simplified (not every service-to-service call is drawn in) - the focus is on the structural relationships: clients → gateway → business services, control flow via the registry, data flow via event bus/shared DB/storage.

flowchart TB subgraph Clients["Clients"] UI_User["User Web-UI"] UI_Admin["Admin Web-UI"] UI_Review["Reviewer/Approval-UI"] UI_Migration["Migrations-Konsole"] CLI["CLI-Tool"] end GW["API-Gateway / BFF-Schicht"] subgraph Control["Control Flow"] REG["Registry Service
(Discovery + License + Sensors)"] ORCH["Plugin Orchestration Service
(Placement, Load Distribution)"] LIC["License Service"] FLEET["Fleet

Management
(multiple installations, optional)"] end subgraph Core["Core Business Services"] AUTH["Auth Service
(AD + Local)"] PERM["Permission Service
(RBAC, Inheritance, Four-Eyes)"] DOC["Document Service
(CRUD, Version, Lock)"] FOLD["Folder Service"] OBJ["Object-Type Service"] RULE["Rule/Constraint Engine"] WF["BPMN Process Engine"] CASE["Case Service (circulation folders)"] end subgraph Support["Supporting Services"] AUDIT["Audit Service"] REPORT["Reporting Service
(Standard Reports + Forensic Trace)"] SEARCH["Search Service"] RENDER["Rendering/Preview Service"] AV["Virus Scan Service"] OCR["OCR Service"] SIG["Signature Service"] NOTIFY["Notification Service"] MIG["Migration/Transfer Service"] CFG["Config Import/Export Service"] QUERY["Query/Trace Service"] end subgraph Federation["Federation (outside the installation)"] HUB["Federation Hub Service
(Address Book + Switching Center)"] end subgraph Connectors["Connector Services"] CMIS["CMIS Connector"] WEBDAV["WebDAV Connector"] CUSTOM["Custom Connector(s)"] end subgraph StorageLayer["Storage Abstraction"] STOR["Storage Service"] S3A["S3 Backend A"] S3B["S3 Backend B"] NFS["NFS Backend"] end BUS(["Event Bus"]) DB[("Shared DB
(separate schemas)")] subgraph Monitoring["Monitoring (external)"] PROM["Prometheus"] GRAF["Grafana"] CHECKMK["CheckMK"] end UI_User --> GW UI_Admin --> GW UI_Review --> GW UI_Migration --> GW CLI --> GW GW --> AUTH GW --> PERM GW --> DOC GW --> FOLD GW --> WF GW --> SEARCH GW --> REPORT GW --> CFG GW --> QUERY QUERY --> AUDIT QUERY --> REPORT QUERY -.Sensors.-> PROM WF <-- Process handoff --> HUB GW -. query availability .-> REG AUTH -. registers .-> REG PERM -. registers .-> REG DOC -. registers .-> REG WF -. registers .-> REG STOR -. registers .-> REG CMIS -. registers .-> REG WEBDAV -. registers .-> REG CUSTOM -. registers .-> REG REG <-- License status --> LIC LIC <-. multiple installations .-> FLEET ORCH <-- Placement data --> REG ORCH -. observes sensors .-> PROM DOC --> RULE FOLD --> RULE DOC --> PERM FOLD --> PERM WF --> PERM WF --> RULE WF --> CASE CASE --> DOC CASE --> DB DOC --> OBJ FOLD --> OBJ DOC --> STOR STOR --> S3A STOR --> S3B STOR --> NFS DOC --> AV DOC --> OCR OCR --> SEARCH WF --> SIG SIG --> DOC DOC --> RENDER CMIS --> DOC WEBDAV --> DOC CUSTOM --> DOC MIG --> DOC MIG --> FOLD MIG --> PERM AUTH --> DB PERM --> DB DOC --> DB FOLD --> DB OBJ --> DB WF --> DB AUDIT --> DB LIC --> DB AUTH ==> BUS PERM ==> BUS DOC ==> BUS FOLD ==> BUS WF ==> BUS MIG ==> BUS LIC ==> BUS BUS ==> AUDIT BUS ==> SEARCH BUS ==> NOTIFY BUS ==> REPORT AUDIT --> REPORT CFG --> OBJ CFG --> RULE CFG --> WF CFG --> PERM Core -.Sensors.-> PROM Support -.Sensors.-> PROM StorageLayer -.Sensors.-> PROM PROM --> GRAF PROM --> CHECKMK

Legend (informal):

- Solid arrows: direct, synchronous API call (data flow).

- Dotted arrows: registration/query via the Registry Service (control flow) or sensor scraping.
- Thick arrows (`==>`): event publishing to the event bus.
- Database symbol: access exclusively to the respective service's own schema (see 3.1), no cross-schema access shown or intended.

---

12. Market Positioning & Gap List

Comparison of the concept against two established market solutions: **nscale** (Ceyoniq, proprietary EIM/ECM system) and **VIS-Suite** (PDV, proprietary ECM platform for public administration). Goal: to be at least equivalent in core functional scope, and deliberately better at the structural weaknesses of the market solutions (e.g. scaling).

Note: a third system named in the original request ("Ldoc") could not be clearly identified as an established market product and was therefore not included here.

Update P12b-S1: the comparison table has been deepened compared to the first draft - some rows are now phrased more concretely instead of merely noting "not publicly documented". For details, see `PROGRESS.md` "Market comparison (Phase 12b, P12b-S1)".

12.1 Comparison Table

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
Architecture	Cloud-native microservices, shared DB with separate schemas, event bus, plugin orchestration	Modular EIM platform, classically on-prem/cloud-hybrid, not consistently stateless-horizontally scalable	Modular concept (mail room, archive, ...), traditionally grown ECM architecture
Horizontal scaling	Every service stateless, instantiable at	Limited: "cluster" means multiple physical instances against **one shared database**,	Not publicly documented, no horizontal

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
	will, automatic time-profile-aware load distribution via the Plugin Orchestration Service	coordination via JGroups, the first connected instance becomes the "coordinator" - load distribution is **not built in** (manual DNS/reverse-proxy assignment), logical instances abolished since version 10.0, instance count limited by CPU cores. Only the stateless rendition component is advertised as horizontally scalable.	scaling advertised as a core feature
Uninterrupted updates	Explicitly planned **and already actually implemented/ verified live** (parallel operation + drain mechanism, expand/contract for schema changes, see 10.5, P10-S3) - a direct consequence of the stateless architecture	**Not supported**: mixed version states of multiple server instances against a shared database schema are explicitly **not supported** - the operating manual requires that **all** installations on one database always be updated simultaneously. No expand/ contract concept.	Not publicly documented as a core feature
Fleet-wide	Explicitly	**No dedicated tool** - neither	Not publicly

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
update orchestration (many installations simultaneously, staggered)	planned as an extension of the fleet management layer (3a, "fleet update orchestration") - reuses the drain mechanism (10.5), scope locks/not-shutdown (4.7/ 4.8), and four-eyes (4.3) per installation	cross-tenant update orchestration nor staggered/ canary rollouts across multiple installations planned as a product feature; operators of larger installation counts must in practice build their own tooling layers for this (staged rollout waves, persistent state machine per stage, structured error classification)	documented
Auditability/ audit-proof compliance	Structurally enforced via the event bus, hash-chain concept, TR-ESOR as reference	Audit-proof archiving present, extensible for TR-ESOR-compliant long-term archiving; no hash-chained/tamper-proof log known (only viewable log files)	Legal hold, access/ modification protection provided, long-term storage via external partners
Workflow/ process automation	BPMN 2.0 **+ DMN 1.3 decision tables** (Update P12b-S1/user decision, see 7.1), modeling +	No-code process platform (PAP) for business departments - BPMN 2.0/DMN 1.3 engine (Camunda lineage), own browser-based modeler, pre-built circulation-folder/ message/follow-up process	Process automation a core component of the current product generation

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
	import, constraint-coupled, configurable four-eyes principle, multi-instance/ federated process steps	templates, multi-stage approval principle preventing duplicate approval by the same person - however, no federated/cross-installation process steps or end-to-end encryption known	
Permissions	RBAC with inheritance, per-folder, scope locks (4.7), domain-separated admin roles (4.6), generic four-eyes principle configurable per action type (4.3)	Noticeably flatter at the infrastructure admin level: access to the administration tools = a single global boolean permission or a few independent checkboxes per account - no resource hierarchy with inheritance, no scope locks, no domain-separated admin roles. A "four-eyes principle" present there is exclusively administrative config-change control (a second account must approve configuration changes), not a generic approval mechanism applicable to arbitrary action types as here	Access-tenant concept, established in the public-sector context
AI functions	Not planned (candidate for later extension, see 12.2)	AI chat, automatic summaries, AI-assisted process support announced (optional plug-in in the web client)	AI integration part of the current product generation

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
Mobile clients	Not planned (web UIs only, see 12.2)	Own iOS/Android app: viewing/ editing, workflow approvals, full-text/geo search, offline access with local encryption	Own mobile client for location-independent file access
Office integration (native editing + metadata + workflow from within Word/ Excel/Outlook)	Explicitly planned (Update P12b-S1/user decision), **cross-platform beyond the core functional scope of some reference products**: Office add-in (Office.js, Windows/Mac/ Web) AND an equivalent LibreOffice/ OpenOffice UNO extension (Windows/ Linux/Mac), see 3.3a	Native ribbon add-in for Word/ Excel/PowerPoint/Outlook - opening/saving without a DMS client, starting/continuing a workflow directly from the document, editing metadata inline, central role-based text-block/template library, email archiving with attachment rules - Windows-/COM-bound, no Linux	Covered via standard processes in the e-file environment
E-signature	Explicitly planned (SES/ AES/QES per eIDAS, broker in front of external	Standard function (embedded/ detached CAdES, verification e.g. via Governikus)	Covered via standard processes in the e-file environment

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
	QTSPs, PAdES/ XAdES/CAdES, see 3.10)		
OCR/text recognition	Explicitly planned, a fixed part of inbound capture, plugin-based engine selection (see 3.9)	Core component of inbound capture (rendition server, single-language per run)	Classic scan/ capture functions established
ERP/business-application integration	Only a generic connector architecture (CMIS/WebDAV/ custom), no pre-built standard connectors	Bidirectional DATEV interface, SAP S/4HANA certified	Integration into business applications/ e-government base infrastructures as a standard requirement
Storage redundancy (multiple backends simultaneously)	Explicitly planned, granularly configurable (write strategy/ quorum) - a distinguishing feature	Structurally weaker: mirroring between two storage backend instances - in pure backup-mirroring mode, failover is **manual** (clients must be switched to the backup server by hand), no quorum-/config-driven write behavior	Not publicly documented at this level of configurability
Configurable monitoring (sensor level, standard	Explicitly planned (Prometheus/ Grafana/	Own monitoring console with traffic-light status per sensor, own history database, email alerts, and Nagios/JMX bridges -	Not publicly documented

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
exporters)	CheckMK)	no Prometheus/Grafana approach known	
Custom object types + free constraints	Explicitly planned (freely modelable)	Categorization by custom criteria, constraint depth not documented	Rather a predefined file-plan/ records-object structure
Circulation folders & case files	Explicitly planned (reference structure, archivable, process-specific processing copies, see 2.3)	File formation established in the e-file environment - a dedicated, pre-built "circulation folder" process template with approval/ acknowledgment/task subprocesses confirms the same basic idea	Core concept of the e-file in the public-sector environment
Mail room/ special areas (inbound/ outbound mail, trash including a classified-documents variant, quarantine, contacts, records-disposal access, templates)	Explicitly planned as its own areas, separated from the regular folder tree, with their own access regime (see 2.5)	Mail room/inbound module established	Core component of the classic records-management module concept
Deletion:	Explicitly	Structurally weaker: deletion is	Not publicly

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
unified model + mandatory deletion reason	planned (unified trash/ mandatory deletion with configurable mandatory deletion-reason requirement, see 5.2/5.2a)	two-staged and spread across two components (the server-side "physical" deletion only removes the record, the actual stored content is separately marked as deleted and only actually released by a separately configured follow-up job) - no unified deletion process, no mandatory deletion-reason requirement known	documented
Accessibility/ failover resilience (alternative representations)	Explicitly planned (automatically generated alternative formats, see 2.4)	Not publicly documented as a standalone feature	Not publicly documented as a standalone feature
Multiple installations (reseller scenario)	Explicitly planned (isolated installations + fleet management)	Not advertised as a core feature	Not advertised as a core feature
Configuration delta between installations	Explicitly planned (baseline-instance comparison, configurable scope, ignore regex for	**No dedicated tool** - no delta/ drift detection between two installations planned as a product feature; operators who need this must in practice build such a comparison tool themselves (configuration diff, criticality-ranked report)	Not publicly documented as a standalone feature

Area	This Concept	nscale (Ceyoniq)	VIS-Suite (PDV)
	naming deviations, see 7.5)		
License model	Multi-dimensional, technically enforced via the registry/License Service, "unlimited" possible per dimension	Market-standard module-based, tiered by usage volume for partial products, file-based license import per component/ host	Module-/user-based, low public level of detail
Open source	Yes (planned)	No, proprietary	No, proprietary

12.2 Candidates for later extension (deliberately not part of the current concept)

These items are identified gaps relative to the market environment, but are not part of this concept version - the architecture (plugin/connector principle, 3.3/3.8) is designed so that they can be added later without intervention in the base framework. **E-signature and OCR have since been moved from this list into the core scope (see 3.9/3.10), as have, in the P12b-S1 plan approval, Office integration (3.3a), DMN decision tables (7.1), the team workspace (2.5), and the extended full-text search query language (3.7a)** - remaining candidates in order of established priority (descending):

1. **Prebuilt standard connectors to common ERP/specialized application software** (e.g., DATEV, SAP) - technically possible via the generic connector architecture, but not preconfigured.
2. **Native mobile clients** (only web UIs planned so far) - typical reference products in the market offer their own iOS/Android app (viewing/editing, workflow approvals, full-text/geo search, offline access with local encryption).

3. **AI functions** (e.g., document chat, automatic summarization, process support) - conceivable as an optional, addable service; lowest priority among the remaining candidates, consistent with the decision in 5.4 to keep AI-supported evaluation currently out of scope.

P14-S3 addendum: for all three remaining candidates, it was verified against the actual code where they would dock into an already existing location, without implementing them - see `docs/extension-points.md`.

---

13. Open Points for the next round of concretization

- **Technology selection - status of decisions**: DB **Postgres** (decided), frontend framework **React/Next.js, predominantly client-side rendering (CSR) to avoid an additional Node SSR runtime layer** (decided, see 8), backend language **Python/FastAPI as the starting point, replaceable service-by-service by other languages thanks to consistent API coupling** (decided, see 1a, including concrete library recommendations for storage/event-bus/auth integration), event bus **NATS JetStream as the standard, Kafka as an option for large installations** (recommended, see 3.4), auth protocol **OIDC based on Keycloak** (recommended, see 4.4), query console parser **pglast/libpg_query** (decided, see 1a/6.1), BPMN engine **SpiffWorkflow** with `bpmn-js-spiffworkflow` as the editor base (recommended, see 1a/7.1).
- Reporting: exact catalog of standard reports (business prioritization by stakeholders needed).
- Mandatory deletion after retention period (5.2a): the factory setting for whether the deletion reason is optional or mandatory should be finally determined (currently left configurable, no system default) - a business decision, ideally made together with the first target customers/object types. Also open: the concrete default lead time for the optional deletion reminder, in case a system default makes sense there instead of completely free configuration.
- Backup & restore (10.4): concrete backup technology per storage backend type (e.g., object versioning vs. a separate backup target for S3-compatible backends, snapshot mechanism for NFS), the exact backup interval for the deletion register (continuous/near-real-time is recommended, concrete technical implementation still open), as well as the test interval for the planned regular restore tests.

- **Uninterrupted updates (10.5) - decided and implemented for the single installation** (P10-S2/S3): a standalone script + registry drain state instead of direct Kubernetes/ OpenShift rolling-update resources (the project only has Docker Compose as a real target system anyway, see P10-S0 finding), canary switchover for individual services verified live. Only the **fleet-wide** level remains open (multiple installations simultaneously, staggered) - see the new "fleet update orchestration" in 3a (P12b-S1 addition) and Phase 13.
- Query & trace console (6.1): concrete extension grammar for the DMS-specific additional constructs based on pglast/libpg_query (parser base is decided, see 1a). The concrete list of tables/object classes to be marked "critical" with mandatory four-eyes principle (6.1) can only sensibly be created after the concrete database schema (table structure per owner schema) has been established - deliberately deferred, not a technical obstacle.
- Plugin Orchestration Service (3.8): the placement algorithm is fundamentally decided (platform scheduler preferred, First Fit Decreasing as fallback) - what remains open is the exact level of detail of the load-profile format (which time-profile categories are concretely distinguished) as well as the exact weighting between time-profile grouping and pure resource size within the FFD procedure.
- BPMN workflow engine (7.1): SpiffWorkflow is LGPLv3-licensed - unlike the building blocks chosen so far (consistently Apache-2.0/MIT-like), so before final determination it is worth a brief legal review of whether the dynamic integration as its own service (no direct linking into a larger compilation unit, since it is operated as an API-separated microservice anyway) is compatible with your open-source licensing goals for the overall system. Purely technically, the separation as a standalone service (instead of a linked library in the same process) is unproblematic, since all services communicate only via APIs anyway.
- Renditions (2.4): concrete list of which target formats are maintained by default per source format (business prioritization, e.g., whether video transcription comes from the start or only as a later extension step) as well as the choice of concrete transcription/conversion libraries/services.
- Circulation folders (2.3): reference behavior is conceptually decided (dynamic reference during processing, fixed closing snapshot at the end of the process) - what remains open is the exact technical implementation of the "dynamic reference" in the data model (e.g., foreign key to the document resolved at runtime vs. trigger-/event-based tracking), should be decided together with the database schema.
- Enforced object hierarchy (2.2a): open whether a subsequent change to `allowedParentTypes` on a class already used in production is checked retroactively against existing filings (blocking/reporting existing, now "impermissible" placements) or

only applies to future new creations/moves - a business decision that should be made together with the first target customers/class schemas rather than abstractly in advance.
- Class icons (2.2a/2.2b): exact origin/format still open - a curated icon set to choose from, free SVG upload, or both (curated set as default, upload as an extension) - a security check (SVG can carry active content) would additionally need to be clarified in the case of free upload.
- Form layouts (2.2b): the grid model (rows/columns, responsive breakpoint) is conceptually decided, the exact data format of the layout JSON (e.g., how field widths per column, nested groups, or conditional visibility of individual fields are represented) is a matter for the respective implementation session.
- Storage device swap (3.6): the exact storage format/procedure of the identity file as well as the prioritization/throttling of background replication after a degraded start (e.g., throttled during core working hours, full speed outside of them) are implementation details of the respective session, not a concept decision.
- OCR configuration (3.9): the concrete unit/estimation method for the word upper limit (e.g., estimated from page count/image resolution vs. exact upfront analysis) as well as a sensible default batch size are implementation details, not a concept decision.

---

14. Preconfigured Configuration Packages (Update P13-S0 lead-in/user request)

Configuration import/export (7.3) already makes it possible to distribute a complete system configuration as a JSON document. This section extends this principle into a standalone product building block: curated, versioned **configuration packages** that put a fresh installation directly into a state ready for use for a specific use case, instead of starting from a completely empty configuration. Deliberately positioned as the **last** substantive extension of the roadmap: a sensible package presupposes that the functional areas it is composed of (object types, workflows/DMN, roles, four-eyes configuration, business calendar, special areas) already fully exist.

14.1 Basic principle: package = curated configuration document + manifest

- A configuration package technically builds **entirely on the existing configuration export/import (7.3)** - no new transfer mechanism, just a curated, maintained compilation of a document format that is already supported anyway.
- A package consists of the actual configuration document (7.3 format: object types, workflows/DMN, role templates, four-eyes configuration, business calendar, optionally structure templates from 2.5) plus a lean **manifest** (package name, version, compatibility range with the system version - analogous to the version-compatibility specification that already exists for federation, 7.4 -, short description, origin/license).
- Applying a package runs through the already existing, secured configuration import (7.3) - no separate permission or approval mechanisms are needed, the same protective mechanisms (e.g., four-eyes, if configured) apply unchanged.
- Packages are intended to be **additive and repeatedly applicable**: the application uses the same upsert logic as a regular configuration import (objects of the same name are updated, not duplicated) - a package can therefore also be applied retroactively to an already running installation that is partly configured differently, not only during initial setup.
- Packages are maintained as standalone, versioned artifacts (e.g., a separate directory/ repository per package) - a marketplace or distribution mechanism is deliberately **not** part of this concept, only the format and the application path.

14.2 eGov configuration package (first concrete package)

First package maintained by the project itself: a sensible default configuration for German public administration (e-government/digital records), so that a fresh installation is directly usable as a ready-to-use eGov DMS after application, instead of first having to manually recreate each of the following settings individually. Deliberate goal: to ship behavior that other market products in this environment **rigidly prescribe** (not switchable off), here instead as a **changeable default setting** - the underlying architecture remains generic in any case, the package only turns it into a sensibly pre-populated starting point:

- **File plan object-type hierarchy**: prepared object types for the structure already mentioned as an example in 2.5 (department → file plan → file), including enforced hierarchy (2.2a) and sensible mandatory attributes (file reference number, file title, lead

department).

- **File reference number format**: preset reference-number format following common administrative convention (e.g., `{Abteilung}-{YYYY}-{Laufende_Nummer}`), year-based counter reset (already existing mechanism, see P5e sessions).
- **Retention period suggestions**: sensible default retention periods per prepared file type (5.2), as a changeable preset, not as a system requirement - the concrete statutory period remains the responsibility of the installation, depending on the federal state/legal area.
- **Classified-documents classification**: prepared attribute/object-type feature with the common German classification levels (VS-NfD, VS-VERTRAULICH, GEHEIM, STRENG GEHEIM), coupled to the already planned classified-documents trash (2.5) and the associated, personnel-separable deletion administration (4.6).
- **Mail room role template**: prepared role assignment for inbound/outbound mail (2.5) corresponding to a typical mail-room organization.
- **Process templates**: prepared BPMN process definitions for common administrative circulation-folder patterns (approval, acknowledgment, task - see 2.3/7.1), as directly usable starting points instead of having to model them yourself.
- **Four-eyes preset**: activated four-eyes principle (4.3) for the action types typically particularly sensitive in the administrative environment (permanent deletion, permission change, configuration import) as the package's default setting - still freely changeable per installation, not a system requirement.
- **Business calendar**: prepared business calendar with the nationwide public holidays as well as, where clearly attributable, state-specific public holidays for SLA deadline calculation (7.1).
- **Extended admin roles**: additional roles modeled on typical administrative organization (e.g., registry/records administration, department head) above the already existing technical domain admin roles (4.6) - pure RBAC configuration via the existing mechanism, not a new one.

14.3 Further conceivable packages (not part of this concept version)

The format is deliberately kept generic so that further industry-/application-specific packages can be added later (e.g., for healthcare, financial services, educational institutions) - not part of this concept version, since it can only be modeled speculatively without a concrete target customer.